The candidate step, higher order: one step over a category of frames, its transport, observers, models and calculi

November 2026

Abstract

The transform calculus has one candidate step. Its states live at declaration frames; a firing moves a state along a morphism of frames; admission is semantic solvability over every later frame, with guards and predicate queries asked at every solution. This note states the step once, as an interface, and records what is proved about it. Frames may be dependently typed: formed telescopes with conversion as their equivalence are an instance whose action laws hold as equalities, the second-order frames translate into them by raising, and the one defect of that translation is η, which typed η restores. Symbolic solvability cannot see empty types; a separate ask over closed solutions can. Firings move along isomorphisms of states up to an equivalence of firings, so transport works up to conversion, and lawful residuals make independence a property of the step. Independence is a diamond that closes up to isomorphism; trace equivalence of dependent runs is a congruence whose quotient is a category, and cells keep what pairs lose. Observation is relative to what observers recorded: equivalence modulo the knowledge is fully abstract under a principal answer, the renamings an observer cannot see are the automorphisms of its record, no deterministic equivariant rule resolves a symmetric choice, and races separate the step's branching identity from its traces. The step has an independent set model, a sound and complete proof logic for its first-order guards and predicate queries, and consistency by a model. Typing contracts, SLD resolution, ground superposition with a complete given-clause loop, bounded search with three outcomes, rational trees, conserved charges with symmetry-quotiented folding, and transfinite runs are instances or readings of the same step. Every result stated as proved names its Lean declaration and module; what is not proved is marked open.

Conventions. Declarations are cited by name together with the module that holds them. Module names are relative to the namespace `Mettapedia.Languages.ProcessCalculi.ConstraintMultisetRewriting`; the step's own modules lie under `Step`, and modules whose names begin with `OSLF` or `TypeTheory` are relative to `Mettapedia`. A state's present occurrences form its *space*; a rule's *patterns* say what a firing takes, reads and makes, what it tells the knowledge, and what it guards. Where a result needs a hypothesis beyond the standard axioms, the hypothesis is named with the result.

Contents

1 Frames and their solutions **2**

2 The step **3**

3 Fibres **4**

4 Dependent and η frames **4**
4.1 The instance . . . 4
4.2 Helpers and their diamond . . . 5
4.3 Symbolic solvability does not see empty types . . . 5
4.4 The non-dependent part is a fibre . . . 5
4.5 The second-order frames translate by raising . . . 6
4.6 The η verdict . . . 6

5 The two presentations are instances **7**

6 Transport **8**
6.1 Lawful transport . . . 8

7 Independence, traces and cells **9**
7.1 Cells . . . 10

8 Footprints indexed by solutions **11**

9 Deciding independence **11**

10 Typing as firings **12**

11 Identity and observation **13**
11.1 Private renaming and escape . . . 13
11.2 Rows modulo the knowledge . . . 14
11.3 Observer records and escape-relative transport . . . 15
11.4 Symmetry . . . 16
11.5 Races and the linear, branching and causal spectrum . . . 16

12 Models **17**

13 Calculi as firings **18**
13.1 Resolution . . . 18
13.2 Superposition . . . 19

14 Search **21**

15 Rational trees and the occurs-check policy for types **22**

16 Charges, folding and the affine pump **23**

17 Transfinite runs **24**

18 Related work **25**

19 What is proved and what is open **26**
19.1 Proved . . . 26
19.2 Corrections to earlier statements . . . 28
19.3 Open . . . 28

1 Frames and their solutions

Definition 1 (Category of frames acting on terms; `Frames` in `Step.Interface`)**.** *A* frame action *consists of frames $a, b, \ldots$, terms T_a at each frame, morphisms $f : a \to b$, an equivalence $\approx_a$ on*

each T_a, an action $f_* : T_a \to T_b$, identities and composites, with $(\mathrm{id}_a)_* t = t$, $(g \circ f)_* t = g_*(f_* t)$, and $s \approx t \Rightarrow f_* s \approx f_* t$.

For an equation store P at a and $f : a \to b$, write $f \models P$ when $f_* s \approx f_* t$ for every $(s, t) \in P$. $\mathsf{Solvable}(P)$ holds when some $f : a \to b$ into some later frame satisfies $f \models P$; an observation φ is *asked* by P, $\mathsf{Entails}(P, \varphi)$, when $\varphi(b, f)$ holds at every such solution; $u : a \to m$ is *principal* for P when $u \models P$ and every solution factors through u on the terms of a.

The canonical instance (`Frames.contextual`). Frames are declaration families (identities with metavariable arities over a binding signature); morphisms are contextual assignments; terms are observations; the action is capture-avoiding instantiation.

The one-frame case (`Frames.ofSubstitution`). A substitution action is the category with one frame; solvability, asks and principality are its consistency, entailment and principality (`ofSubstitution_solvable_iff`, `ofSubstitution_entails_iff`, `ofSubstitution_principal_iff`).

Theorem 1 (`solvable_iff_into`, `firstOrderFrame_weaklyTerminal`, `contextual_solvable_iff_consistent` in `Step.Interface`)**.** *Existence is witnessed at a weakly terminal frame, and the family with one parameterless metavariable per sort is weakly terminal: existence never needs a higher-order helper.*

Definition 2 (Isomorphism of frames; `Frames.Iso` in `Step.Transport`)**.** *Morphisms $f : a \to b$ and $g : b \to a$ such that $g_* f_* t \approx t$ and $f_* g_* t \approx t$ for all terms. They form a groupoid (`Iso.refl`, `Iso.symm`, `Iso.trans`).*

Proposition 2 (`Iso.solvable_applyStore_iff`, `Iso.equivalent_symm`, `Iso.equivalent_trans` in `Step.Transport`)**.** *Isomorphisms keep solvability, and stores with the same solutions along an isomorphism stay so along its inverse and along composites. Renaming fresh helpers is such an isomorphism.*

Definition 3 (Closed asks; `Frames.ClosedSolvable`, `Frames.ClosedEntails`, `Step.ClosedAdmits` in `Step.DependentClosed`)**.** *Given a designated closed frame, a store is* closed-solvable *when it has a solution into the closed frame; a* closed ask *holds at every such solution;* closed admission *asks both over closed solutions. Closed and symbolic solvability agree when a principal answer lands at a frame with a closed point, and on every store when the closed frame is weakly terminal (`closedSolvable_iff_of_principal`, `closedSolvable_iff_of_weaklyTerminal`).*

2 The step

Definition 4 (Candidate step; `Interface.Step` in `Step.Interface`)**.** *A state s has a frame, knowledge K_s (an equation store there) and a space W_s of present occurrences. A firing f at s has a selection (taken, read and made occurrences), a target frame with an extension e_f from the state's frame, information I_f containing $(e_f)_* K_s$, shapes, and guards and queries $H_f(b, \sigma)$ at each solution. Admission is characterized once (`admits_iff`):*

$$\mathrm{Admits}(f) \iff \mathrm{Shapes}(f) \wedge \mathrm{Enabled}(W_s, f) \wedge \mathsf{Solvable}(I_f) \wedge \mathsf{Entails}(I_f, H_f).$$

An admitted firing has a result with space $(W_s \setminus \mathit{taken}) \cup \mathit{made}$ and knowledge $(c_f)_* I_f$ along a settling morphism c_f.

Runs are dependent words of admitted firings (`Step.Run`); knowledge only refines along them (`Run.models_start`); they concatenate (`Run.append`); site accounts are functors out of them (`Run.account_append`). The interface itself leaves the settling morphism and the reading of guards unconstrained, and a transport's residual (Section 6) has only selection and naturality laws; Section 6.1 gives the laws that constrain all three.

3 Fibres

1. **The first-order fibre.** Every metavariable is parameterless. Unification decides consistency and returns principal answers (`FirstOrder.accepted_iff_consistent`, `FirstOrder.accepted_principal` in `FirstOrder`).
2. **The pattern fibre.** Every metavariable occurrence is applied to distinct bound variables. The structural pattern driver always finishes with a principal answer or a certified refutation (`ContextualPatternTermination.finishes`, `solves_or_refutes`, `refutes_of_no_model` in `ContextualPatternTermination`).
3. **Deferred residuals elsewhere.** A non-pattern equation stays a residual constraint; every solution survives each bounded run of the driver (`ContextualFrameWorklist.preserves_every_solution` in `ContextualFrameWorklist`), and a reduction presents exactly the original solutions (`ContextualResidual.Reduction` in `ContextualResidual`). Latent obligations of conditional rows enter the information of the firing that selects them. A solvable flex–flex equation is never refuted by any run of the driver (`FlexFlex.never_refuted` in `Step.BetaEtaReadings`), so refusing it would be unsound; general second-order unification is undecidable [27].
4. **Beyond second order.** The second-order frames translate into the dependent instance of Section 4. The rational-tree fibre of Section 15 is a sibling of the first-order fibre, not a fibre of the dependent instance.

Canonical forms. The framework is second order: metavariable parameters have sorts, so instantiation is hereditary substitution and never creates a framework redex. Instantiation equals the evaluation of the explicit redexes that lazy instantiation writes (`evaluate_lazy`), canonical forms represent framework conversion uniquely (`convertible_iff`, both in `OSLF.Syntax.HereditaryBetaEtaFramework`), and the pattern driver therefore unifies modulo framework $\beta\eta$ (`models_iff_framework_convertible` in `Step.BetaEtaReadings`). Object-level $\beta\eta$ is a step unless a conversion theory is admitted. An admitted theory is its own frame category, in which the syntactic driver's refutation is unsound (`refutation_not_admitted`) and reading the head of stored data is not well defined (`head_readout_not_invariant`). A metavariable whose parameter binds would make the framework third order, where hereditary substitution cascades; metavariable arities exclude it. *Open:* solvers for admitted object theories. Models of these fibres are in Section 12.

4 Dependent and η frames

The interface asks of frames only objects, terms, morphisms, an equivalence and action laws. A dependently typed instance therefore needs no change to it, and the smaller settings it was built to contain are fibres of it: the non-dependent part is a full sub-instance on the frames that satisfy a predicate, and the second-order frames are the image of a frame map.

4.1 The instance

Theorem 3 (Dependent frames; `Dependent.frames`, `frames_models_iff`, `pointwise_iff`, `solvable_empty_iff` in `Step.DependentFrames`)**.** *Over a rule package of a formation-sensitive calculus with dependent function, pair and identity types and a universe tower, frames are formed telescopes, morphisms are formed substitutions, terms are judged terms (a formed type with a term of it), the action is substitution on both components and the equivalence is conversion of both. The action laws hold as equalities. Two solutions act alike exactly when they are pointwise convertible, and at the empty frame a store is solvable exactly when every equation already converts.*

A metavariable $M : (\Psi \vdash A)$ is reached by raising [41]: it is a telescope variable of type $\Pi\Psi.\, A$. A definition $x \doteq v$ has the principal answer $x := v$, principal answers extend along

definitions, and asks of equality guards are decided there (`principal_definition`, `definition_principal`, `definition_entails_iff` in `Step.DependentFrames`). An ill-typed equation has no solution in any later frame, because solutions convert types too (`Consumer.illTyped_unsolvable` in `Step.DependentControls`).

4.2 Helpers and their diamond

Theorem 4 (`Dependent.helperStep`, `admits_iff`, `helper_closes`, `independent_iff`, `swap_fixes`, `swap_exists_iff_fixed`, `no_swap_of_not_weakened` in `Step.DependentHelpers`)**.** *A helper firing makes one occurrence and one variable $x : A$, with A formed over its frame, and tells $x \doteq v$. It is admitted exactly when the knowledge is solvable, its occurrence is fresh and the knowledge asks its guards. Two helper firings close up to the swap $\Gamma.B.A \cong \Gamma.A.B$, which fixes every old variable, and independence is solvable knowledge, fresh and distinct occurrences, and both guards asked. A run $f; g_1$ swaps exactly when substituting f's value for its helper leaves g_1 unchanged; if g_1's type is not a weakening, no swap exists.*

The helpers $x : g \to g := \lambda x.x$ and $y : \mathrm{Id}\,(g \to g)\,(\lambda x.x)\,(\lambda x.x) := \mathsf{refl}$ are independent, and their composites differ and are isomorphic (`Consumer.independent`, `Consumer.composites_differ` in `Step.DependentHelpers`). The helper $y : \mathrm{Id}\,(g \to g)\,x\,x$ after x forms a run with no swap (`Consumer.dependent_run_has_no_swap` in `Step.DependentControls`). A helper whose type is formed only through conversion is admitted (`Consumer.converted_admitted`).

4.3 Symbolic solvability does not see empty types

Theorem 5 (`solvable_extend_iff`, `closedSolvable_extend_iff`, `obligation_trap`, `closedAdmits_iff_admits`, `trap_helper` in `Step.DependentClosed`)**.** *Knowledge read at $\Gamma.A$ is solvable exactly when it is solvable at Γ, for every type A, empty or not; closed solvability at $\Gamma.A$ needs a closed term of A at some closed solution. At an obligation $x : A$ the empty store is always solvable, and closed-solvable exactly when A has a closed term. For guard-free firings, closed and symbolic admission agree when the knowledge has a principal answer at a closed-inhabited frame; at an obligation of a type with no closed term, every fresh guard-free helper firing is admitted and none is closed-admitted.*

Over the universe tower, $\bot := \Pi(X : U_0).\,X$ is formed, and at the frame $x : \bot$ the helper $y : \bot := x$ is admitted and not closed-admitted (`TowerTrap.trap` in `Step.DependentClosedTower`). Emptiness of $\bot$ is proved from an explicit hypothesis, cofinally many inaccessible cardinals, so this control is a relative-consistency statement. At $x : g \to g$, which $\lambda x.x$ inhabits, a helper is both admitted and closed-admitted (`Consumer.copy_admitted` in `Step.DependentClosed`). Consistency of constraints and inhabitation are therefore two asks. In a dependent foundation a metavariable is a proof obligation, and symbolic admission alone would accept a hole of an empty type.

4.4 The non-dependent part is a fibre

Theorem 6 (`Frames.restrict`, `simpleStep_admits_iff`, `principalInside_after`, `admits_iff_along` in `Step.DependentFibre`; `packing`, `simple_retracts`, `simpleStep_admits_iff_of_joins`, `tower_fibre` in `Step.DependentPacking`)**.** *On the full sub-instance of telescopes none of whose entries mentions a variable, admission computed in the sub-instance equals admission in the full instance whenever the knowledge has a principal answer at such a frame. That holds from empty knowledge and is kept by every firing, so the fibre decides admission exactly along every run from empty knowledge. Every formed telescope is a retract, up to β for pairs, of a frame with at most one variable of a closed Σ-type; when universe levels have joins, as in the universe tower, the fibre theorem holds for every store.*

4.5 The second-order frames translate by raising

Theorem 7 (`instantiate_beta`, `action_beta` in `Step.TranslationSyntax`; `Translation.translation`, `translation_comp` in `Step.TranslationFrames`; `translation_models_iff`, `translation_reflects`, `translation_entails_reflect` in `Step.TranslationReflection`)**.** *The translation sends a metavariable of arity* (Γ, s) *to a variable of type* $\Pi\Gamma.\, s$*, an occurrence* $m(\bar{a})$ *to that variable applied to* $\bar{a}$*, and an assignment to a substitution of closed abstractions. It is a frame map from the second-order instance on finite ordered frames into the dependent frames; contextual instantiation is* β *on these applications, and composition is preserved on every judged term. Over a rule package with no root computation, symmetric head equality and injective sort and operator names, translated terms are normal and the encoding is injective; dependent solutions of a translated store are exactly the translations of second-order solutions, and asks over all dependent solutions reflect to second-order asks.*

Admission transfers when the information has a second-order principal answer whose translation factors every dependent solution on translated terms (`admission_transfer`), and that hypothesis holds for every definition $m(\bar{x}) \doteq t$ with m outside the support of t (`definition_admission_transfer` in `Step.TranslationAdmission`). For arbitrary stores it needs adequacy: every dependent solution of a translated store must factor through a second-order one on translated terms. That is open.

4.6 The η verdict

Theorem 8 (`identity_code_not_variable` in `Step.TranslationReflection`; `pattern_no_principal`, `pattern_principal`, `translation_not_preservesPrincipal` in `Step.TranslationPrincipal`; `pattern_entails_iff` in `Step.TranslationImage`; `EtaFrames.etaFrames`, `EtaFrames.eta_pattern_principal` in `Step.TranslationEta`)**.** *Without* η *the translation has exactly one defect. The code* $\lambda\bar{x}.\, m\,\bar{x}$ *is not convertible to* m*, and over every root-free, head-symmetric package the translated pattern equation* $m(x) \doteq n(x)$ *has no principal answer in the dependent frames: the solutions* $m \mapsto g$*,* $n \mapsto \lambda x.\, g\,x$ *and* $m \mapsto \lambda x.\, g\,x$*,* $n \mapsto g$ *cannot both factor through one answer, while the second-order instance has a principal answer. On translated terms nothing is lost, since they observe a metavariable only through full applications, and asks transfer for the pattern equation. Over typed definitional equality with* η *for* Π *and* Σ*, the translated second-order answer is principal again.*

The canonical η instance. It is `ContextualDependentEta.frames` in `ContextualDependentEta`: formed contexts of the typed contextual category with typed total terms, whose equivalence is typed equality with dependent function and pair η (`ContextualDependentEta.function_eta`, `ContextualDependentEta.pair_eta` in `ContextualDependentEta`). Every solution of the dependent frames of a qualified computation package (constructor forms, root and head preservation, Church–Rosser for raw conversion) is an η-solution (`BetaBridge.models`, `BetaBridge.solvable` in `ContextualDependentEtaBridge`). A function against its η-expansion and a pair against its projections have η-solutions and no solution in the dependent frames (`Controls.function_eta_solved`, `Controls.function_requires_eta`, `Controls.pair_requires_eta` in `ContextualDependentEtaControls`). An executable type-directed comparison scans equation stores while keeping every later solution and every ask (`Conversion.scan_models`, `Conversion.scan_solvable_iff`, `Conversion.scan_entails_iff` in `ContextualDependentEtaConversion`). The principal answer for $m(x) \doteq n(x)$ above is proved over a sibling η instance on raw telescopes at one universe (`EtaFrames.etaFrames` in `Step.TranslationEta`); stating it over the canonical instance is open.

Verdict. The foundation is dependent frames whose equivalence is typed equality with η; the second-order step is their η-long fibre. The property that made the second-order base attractive,

principal answers for patterns, comes from η, not from being second order. The opposite design is the awkward one: written as an object theory inside the second-order base, dependent function, pair and identity types make conversion a rewriting, and the equation $(\lambda x.x)\, y \doteq y$ then has no solution (`ObjectTheory.object_beta_unsolvable` in `Step.TranslationObjectTheory`).

Native type theory as the semantics. Every strict category with families is a frame instance (`NativeUniverse.cwfFrames`), and the displayed-presheaf category with families gives one whose action laws hold by reflexivity (`NativeUniverse.nativeFrames`, in `Step.NativeUniverseFrames`); its Π validates β and η (`pi_beta`, `pi_eta` in `TypeTheory.DisplayedPresheafPi`). This presheaf universe of native type theory [61] is the semantics of the frames, not their checking base. A conversion-sound map from the dependent frames into it is open.

Toward a dependent solver. A bounded conversion scan keeps both endpoint types, leaves unresolved equations pending and preserves every later solution and guard (`ContextualDependentConstraints.scan_models`, `scan_solvable_iff`, `scan_entails_iff` in `ContextualDependentConstraints`); eliminating the newest defined variable preserves and reflects solutions (`eliminateDefinition_models_iff`, `eliminateDefinition_solvable_iff` in `ContextualDependentElimination`). A driver that finishes with a principal answer or a refutation on the dependent pattern fragment [45, 1] is open.

5 The two presentations are instances

The first-order run layer is the one-frame case (`AnnotatedRuns.firstOrder` in `AnnotatedRuns`). Admission is the run layer's admission and executable words are runs (`firstOrder_admits_iff`, `executable_iff_run`, `follow_eq_last`). Admission, the update and runs are equivariant under label-preserving renamings of occurrence identities (`Firing.valid_rename`, `executable_rename`).

Native firings are the canonical case, with published results (`native` in `Step.NativeInstance`). A native firing's data are its rule, selection, outputs, reuse flags, queries, budget, and its publication *recipes*: for each produced output, the private and captured roles of the names of its emitted term and its local conditions, fixed before execution and absent elsewhere. Admission is the interface's (`native_admits_iff`). An admitted firing pins one execution in the published-execution record (`NativeFiring.pinned`); the record has no row decoder. The published rows publish each recipe's output from the actual emitted term, and keep every other row (`publishRows`); the published state keeps the execution's frame, information and occurrences (`publishedState_declarations`, `publishedState_equations`, `publishedState_present`).

Theorem 9 (`native_published`, `publishedState_none`, `native_raw_target`, `publishedState_eq_rawTarget_iff` in `Step.NativeInstance`)**.** *The step's result is the published state of every record with the firing's data and budget. Without recipes it is the raw target. The published state is the raw target exactly when every recipe republishes its output's raw row.*

Example 1 (`template_published`, `PublicationControls.published_differs_from_raw_target`, `PublicationControls.native_result_depends_on_recipes` in `Step.NativePublicationControls`). A direct template's recipes publish its execution as the template's own target. At an actual mixed template execution, at every budget, the published state differs from the raw target: an incompatible dormant schema stays unmatched where the raw target holds a matchable shared row. Two native firings with the same execution data, with and without the template's recipes, have different results, so the result is not a function of the raw execution.

Open: native firings of mixed templates, which publish through producer recipes into a packed private family, are not yet recipe instances of the step; native firings have no transport yet.

6 Transport

Definition 5 (Isomorphism of states; `Step.StateIso`, `StateIso.rename` in `Step.Transport`)**.** *An isomorphism of frames that carries the knowledge of one state to knowledge with the same solutions as the other's, together with a bijection of occurrence identities that carries the space of one onto the space of the other. Isomorphic states have solvable knowledge together (`StateIso.solvable_iff`). A term carried along a morphism that agrees with another at every solution of the target's knowledge is* carried up to the knowledge *(`Frames.Carried`); that clause relates what clients read, stored rows and public terms, and it is closed under inverses and composites (`Carried.symm`, `Carried.trans`).*

The interface has no labels and no clients, so which identities a renaming must fix belongs to the layers that have them (Section 11).

Definition 6 (Transport; `Transport` in `Step.Transport`)**.** *An instance chooses a groupoid of isomorphisms of states over the interface's and an equivalence of* alike *firings at one state: firings whose data agree up to the frame equivalence once their frames are identified. Alike firings have one selection, are admitted together and have isomorphic results (`selection_alike`, `admits_alike`, `result_alike`). Firings move along isomorphisms by the functor laws up to alikeness (`move_refl`, `move_trans`, `move_symm`, `move_alike`), select the renamed occurrences, keep their admission, and their results stay isomorphic. The* residual *of f after an admitted g is f read at g's result along g's morphism of frames; it keeps its selection, and residuals are natural up to alikeness (`move_residual`). Where alike firings are equal, the laws are equations (`strict_laws`).*

Theorem 10 (Transport up to conversion; `Dependent.liftConv`, `Dependent.extendIso`, `Dependent.FiringAlike`, `Dependent.conversionTransport`, `conversion_move_strict`, `Consumer.moved_back_ne`, `Consumer.moved_back_alike`, `Consumer.not_alike` in `Step.DependentConversion`)**.** *At dependent frames, a substitution lifts under a variable whose type converts, and an isomorphism of frames lifts under convertible types, with its inverse laws pointwise up to conversion. Alike helper firings make the same occurrence, with convertible type and value and guards convertible equation by equation. These give a transport along every isomorphism of states, which moves exactly as the strict helper transport on strict isomorphisms. Moving $y : \mathrm{Id}\,(g \to g)\,x\,x$ along $x \mapsto (\lambda y.y)\,x$ and back gives a firing that is not equal to the original and is alike to it; firings whose types do not convert are not alike.*

Strict telescope isomorphisms alone do not give the functor laws at dependent frames, because a type moved along an isomorphism and back is convertible, not equal, to the original.

States up to isomorphism form *state classes* (`Transport.StateClass` in `Step.Transport`); the class relation is a congruence for the step: every admitted firing at one state of a class has a counterpart with the renamed selection at every other, and the results are again in one class (`Transport.same_step`, `Same.admits`). *Similar* runs are, firing by firing, alike to transports of each other (`Transport.Similar`); similarity is an equivalence, is compatible with concatenation (`Similar.append`) and with alike prefixes (`Similar.cons_alike`), relates isomorphic starts and ends (`Similar.same_start`, `Similar.same_last`), includes every run's transport (`Similar.moveRun`), and keeps every site account whose grades are invariant under transport and alikeness (`Similar.account_eq` in `Step.Transport`).

6.1 Lawful transport

Three laws, stated over an existing transport, make independence a property of the step rather than of a chosen residual.

Definition 7 (`Step.Guards`, `Step.ConservativeSettlement`, `Transport.Coherent`, `Transport.Lawful` in `Step.LawfulTransport`)**.** Locality of guards*: the guard of a firing at a solution is a predicate of the space that respects the action and reads only a declared region; a firing that writes outside the region keeps the guard (`Guards.guard_fire`).* Conservative settlement*: every solution of a firing's information agrees with one that factors through the settling morphism, so a result asks exactly what its information asks (`entails_result_iff`, `solvable_result_iff`).* Residual coherence*: the residual f/g lives where f's frame and g's result meet, which is a pushout for solutions over the state's frame; its information is f's carried there together with g's result knowledge, its shapes are f's, and its guards are f's read along the carrying morphism. A transport is* lawful *when it is coherent and conservative and* canonical*: isomorphic firings at one state have results in one class.*

Theorem 11 (`Coherent.admits_iff`, `Coherent.firingIso`, `Lawful.independent_iff`, `Lawful.independent_withResidual_iff` in `Step.LawfulTransport`)**.** *A coherent residual is admitted exactly when a condition that names no residual holds; two coherent residuals are isomorphic firings; and two lawful transports with the same state classes have the same independent pairs, so replacing one lawful residual by another keeps independence.*

The first-order transport is lawful when the action is faithful, and it is (`firstOrderLawful`, `firstOrder_faithful` in `Step.LawfulFirstOrder`); its region is the occurrences matched by absence and count queries together with the query regions. The helper transport is lawful with the empty region (`Helpers.helperLawful` in `Step.LawfulHelper`), and so is the rational-tree transport (Section 15). Without the laws, independence depends on the residual chosen: the tells $X = 1$ and $Y = 2$ are independent under the lawful transport and not under a residual that drops tells, which is not lawful (`LawfulControls.unlawful_changes_independence`, `LawfulControls.noTells_not_lawful`). Settling by the fixed substitution $X \mapsto 1$ satisfies every interface law, is not conservative, and its result asks $X = 1$ where the information did not (`LawfulControls.settledBy_not_conservative`, `LawfulControls.settledBy_asks_more`). A non-faithful action has no guard structure at all (`LawfulControls.blind_no_guards`, all in `Step.LawfulFirstOrder`).

7 Independence, traces and cells

Definition 8 (Independence; `Transport.Independent` in `Step.Independence`)**.** *Firings f, g at s are independent when both are admitted, the residual of each after the other is admitted, and the two composite results are in one state class: the diamond closes up to isomorphism.*

A *swap* replaces f then a firing alike to the residual of g by g then a firing alike to the residual of f, carrying the rest of the run along a similar run (`SwapStep`); *trace equivalence* is generated by swaps and similarity (`TraceEquiv`).

Theorem 12 (`swap_independent`, `Independent.swap`, `TraceEquiv.same_last` in `Step.Independence`)**.** *Every swap swaps an independent pair, every independent pair swaps in front of every continuation, and trace-equivalent runs end in one state class.*

Theorem 13 (Congruence; `TraceEquiv.append_left`, `TraceEquiv.append_right`, `TraceEquiv.moveRun`, `TraceEquiv.comp` in `Step.Independence`)**.** *Trace equivalence is kept by prefixing, by appending similar continuations, and by transport along an isomorphism of the start; composition (concatenation after transport) is well defined in both arguments.*

Theorem 14 (The category of dependent traces; `traceCategory` in `Step.TraceCategory`)**.** *State classes and runs between them, up to similarity and trace equivalence, form a category under concatenation after transport (`Arrow.comp_rel_left`, `Arrow.comp_rel_right`, `Similar.append_assoc`, `Similar.append_nil`).*

Theorem 15 (Site accounts; `TraceEquiv.account_eq`, `TraceEquiv.account_eq_comm` in `Step.Independence`)**.** *A site account is a property of traces when its grades are invariant under transport, alikeness and residuals and commute on independent firings; in a commutative monoid, invariance suffices.*

The one-frame instance (`firstOrderTransport` in `Step.FirstOrderIndependence`). Isomorphisms are configurations with the same solutions and occurrences; firings move and have residuals as themselves.

Theorem 16 (`firstOrder_closes`, `firstOrder_independent_iff`, `firstOrder_independent_iff_static`, `firstOrder_account_eq` in `Step.FirstOrderIndependence`)**.** *Diamonds close whenever both orders are admitted, so independence is the four admissions. For firings without guards whose query regions do not depend on the solution, once one order runs, independence is exactly the static independence of the run layer. Site accounts are properties of its traces in every commutative monoid.*

Theorem 17 (Static traces are traces; `similar_of_firings`, `swap_traceEquiv`, `traceEquiv_of_static` in `Step.StaticTraces`)**.** *Runs with the same firings from configurations with the same solutions and occurrences are similar, a swap of statically independent firings in a run is a swap of runs, and runs whose words are statically trace equivalent are trace equivalent: the run layer's trace theory of words is an instance of the trace theory of dependent runs.*

The smallest contextual instance (`Helpers.helperStep` in `Step.ContextualDiamond`). A helper firing makes one occurrence and one fresh helper metavariable and tells the helper's value, a term over the frame it fires at; its guards are equations on the names of that frame, asked at every solution; its result lives at the frame with one more name. A firing moves by instantiating its value and renaming the occurrence it makes, and its result moves along the isomorphism lifted to the extended frames, fixing the helper (`Helpers.liftIso`, `Helpers.information_move`); asks of guards are carried along isomorphisms (`Frames.Iso.entails_applyStore_iff`).

Theorem 18 (`Helpers.admits_iff`, `Helpers.helper_closes`, `Helpers.independent_iff` in `Step.ContextualDiamond`)**.** *A helper firing is admitted exactly when the knowledge is solvable, its occurrence is fresh, and the knowledge asks its guards: the helper's equation never constrains the old names (`Helpers.entails_information_iff`). Firing f then g and g then f reach frames holding the two helpers in the two orders, and swapping them (`Helpers.swapIso`) is an isomorphism of the results that fixes every name of the starting frame (`Helpers.swap_incl_incl`) and renames no occurrence. Independence is exactly solvable knowledge, fresh and distinct occurrences, and the guards of both firings asked by the knowledge.*

Since the closing isomorphism renames no occurrence and fixes the old names, a following firing that refers to occurrences or to old names finds a counterpart with the same selection, the same admission and an isomorphic result at the other composite (`Transport.same_step` in `Step.Transport`); only the two fresh helpers are renamed.

Example 2 (A concrete diamond; `Consumer.independent`, `Consumer.residuals`, `Consumer.composites_differ` in `Step.ContextualDiamond`). At the empty frame, a firing making occurrence 1 and a parameterless helper valued 7, and a firing making occurrence 2 and a one-parameter helper valued 8, are independent; each residual keeps its occurrence, helper and value in the frame the other made; the two composite states are different and isomorphic: the diamond commutes only up to renaming.

7.1 Cells

Pairs lose what rounds keep: the tells $X = Y$, $X = 1$ and $Y = 2$ are pairwise independent but cannot all fire.

Definition 9 (`Transport.Cell`, `Transport.Closing`, `Transport.CubeLaw` in `Step.Cells`)**.** *A* cell *at a state is a round of firings each of which is admitted and after which the residuals of the others form a cell. A* closing *chooses an isomorphism for every admitted square, and the* cube law *says that moving the double residual* $(h/f)/(g/f)$ *along the closing gives* $(h/g)/(f/g)$*.*

Theorem 19 (`cell_pair_iff`, `cell_pair_iff_independent` in `Step.Cells`; `firstOrder_cubeLaw`, `CellControls.hollow_cube`, `CellControls.no_three_cell` in `Step.CellsFirstOrder`; `Helpers.helper_cube`, `Helpers.helperCubeLaw`, `Helpers.swap_braid` in `Step.CellsHelper`)**.** *Two firings form a 2-cell exactly when both orders are admitted; where diamonds close, 2-cells are the independent pairs. In the one-frame instance the cube law holds for every closing, and the three tells above are pairwise independent 2-cells that form no 3-cell. For helper firings, the double residuals of a third firing agree along the closing swap, the cube law holds for closings whose frame maps are the helper swaps and whose renamings are the identity, and the swaps satisfy the symmetric-group relations, including the braid relation for three helpers.*

Conflict is therefore not binary: pairwise independence does not give joint consistency of a round. Lower faces as cells and a precubical set of rounds with face maps are open.

8 Footprints indexed by solutions

Theorem 20 (Sufficient; `round_independent` in `Step.FirstOrderIndependence`)**.** *In the one-frame instance, a round of two is independent: when the members' footprints are independent and each writes nothing into the other's guards and query regions at every solution of the pair's cut, the pair is independent.*

Example 3 (Overlap at one solution only; `TransportControls.overlap_at_one_solution_only` in `Step.FirstOrderIndependence`). A query whose region is the labels 1 when the solution sets $X = 1$, and nothing otherwise, and a writer of label 1: the footprints are disjoint at $X = 2$ and overlap at $X = 1$. The pair is not compatible and not statically independent, and it is dependent; reasoning from the footprint at $X = 2$ alone would accept it. A writer of label 0 forms a round with the query and is independent (`TransportControls.outsider_watcher_independent`).

Theorem 21 (Grounded regions; `principal_entails_iff`, `ask_matching_iff`, `ignoresWrites_matching_iff` in `Step.GroundedRegions`)**.** *A query region that matches a pattern read at the solution is the same at every solution when a principal answer grounds the pattern. Then the universal ask is one evaluation at the answer, and a writer stays outside the region at every solution exactly when none of its labels is equivalent to the grounded instance. In the first-order fibre the unifier's answer serves (`firstOrder_ask_matching_iff`); with the information* $X = 1$*, a maker of* 1 *writes into the region of* X *and a maker of* 0 *does not (`GroundedControls.writer_not_ignored`, `GroundedControls.outsider_ignored`).*

9 Deciding independence

Theorem 22 (`independent_iff_admits`, `decidableIndependent` in `Step.Independence`; `Frames.Principal.entails_iff`, `Principal.entails_models_iff` in `Step.PrincipalAnswers`)**.** *Where diamonds close, independence is the four admissions and is decided wherever admission is. An ask of an observation that respects the action and survives further instantiation is decided at a principal answer; equality guards are such observations.*

Theorem 23 (First-order fibre; `valid_iff_tests` in `Step.FirstOrderIndependence`; `firstOrder_decidableConsistent`, `firstOrder_decidableIndependent` in `Step.GroundedRegions`)**.** *For firings without guards whose query regions do not depend*

on the solution, admission is the shapes, the occurrences, one consistency test and one evaluation per query; unification decides consistency, so independence is decided.

Theorem 24 (Pattern fibre; `Helpers.pattern_solvable_iff`, `Helpers.pattern_unsolvable_iff`, `Helpers.helperCheck_decides` in `Step.ContextualDiamond`)**.** *Pattern knowledge is solvable exactly when the driver returns an empty residual at some budget, and unsolvable exactly when it refutes at some budget. The check of helper independence answers at some budget on pattern knowledge, for firings without guards.*

Definition 10 (Three-valued check; `Transport.Check` in `Step.Independence`)**.** *A check answers independent, dependent or unknown, and never answers either of the first two wrongly (`Check.ofTests`). A check built from a decision never answers unknown (`Check.ofDecidable_ne_unknown`).*

Example 4 (Unknown outside the fragment; `Helpers.helperCheck_unknown`, `NonPattern.independent`, `NonPattern.unknown` in `Step.ContextualDiamond`). The knowledge $M(3) = 3$ applies a metavariable to a constant, outside the pattern fragment, and is solvable by $M \mapsto \lambda x.\, 3$. Two helper firings with fresh, distinct occurrences are independent there, and the check answers unknown at every budget.

Why undecidable unification does not block the definition. Independence is defined by admission, which asks over every solution in every later frame, and by state classes. No definition chooses a solution: the knowledge of a state is a store of residual equations, kept as it is. Where unification is undecidable, independence remains a proposition with the same meaning, and a sound check may answer unknown. Bounded searches give such checks at every budget (Section 14).

10 Typing as firings

Typing contracts of a runtime type system, the reuse of typing evidence, an exit check, typing of states and bidirectional checking are all firings of the step.

Contracts. A typed call is a firing of the one-frame step over a constraint domain whose terms are program values, type expressions and membership atoms, two terms being equal when they mean the same in every ground model. Signature parameters are renamed to fresh type helpers that may take only proper types, which write no reserved improper name such as `Atom`; program variables stay metavariables of the knowledge, so a call never binds them.

Theorem 25 (`call_admits_iff`, `closed_call_admits_iff`, `closed_call_admits_iff_accepted`, `call_admits_iff_residual`, `runtime_accepts_of_admits` in `Step.TypingContracts`)**.** *A call firing is admitted exactly when some point satisfying the knowledge meets the joint membership judgment of its arguments and output. For closed values this is the declaration-based judgment, and, with no reserved names, a closed type and decidable equalities, it is exactly what the existing call checker accepts. For open values, admission is the existence of a binding that satisfies the compiled overloads, and an admitted call is accepted by the runtime's test of the type equations of that residual formula.*

The runtime decides those type equations, and its refusals are refusals of the step; it does not decide whether a pending membership can still be met, which is inhabitation of the requested types. *Controls* in `Step.TypingContractsControls`: `same_mixed_refused` and `same_numbers_admitted` (a shared parameter refuses a number with a string and admits two numbers); `atom_solves_shared_parameter` (if helpers may take `Atom`, the mixed call is admitted); `open_pending`, `contradicting_refused` and `lifecycle_run` (an open argument is admitted pending and a contradicting later equation is refused, in one run); `void_trap` (the runtime accepts a requirement

at an empty type, which the step refuses: the gap of Section 4 again); `guard_refuses_shared_parameter` (reading a closed membership as a guard refuses a call the contract admits, so memberships at shared parameters are told).

Theorem 26 (Evidence; `guard_run`, `member_inside`, `reuse_sound`, `generation_sound`, `fine_subset_coarse` in `Step.TypingEvidence`)**.** *For any step with local guards, a guard reads the same at the end of every run that writes nothing in its region. With declarations as rows, membership reads only the rows declaring a rule for a symbol of the value together with the fact rows, and checked membership stays valid along every run that writes nothing there. A generation counter that invalidates on every declaration edit is sound, because its region contains that one.*

It is also coarse: an edit of an unrelated declaration writes the coarse region and not the fine one, and the membership still holds; retracting the number row writes the fine region, and it fails (`EvidenceControls.unrelated_edit`, `EvidenceControls.related_edit_breaks`).

Theorem 27 (Exit check; `sequential_exit_keeps_effects`, `staged_exit_no_residue`, `staged_exit_commits`, `staged_executable_iff` in `Step.TypingExit`)**.** *When a call's main firing is admitted and its exit check is refused at the result, the main firing alone is a run, no run has both, and everything the main firing made stays present. In a deferred-effects block with the main firing, a refused exit leaves the block's result at the start; when both are admitted the block publishes the composite; deferred and sequential executability agree.*

Theorem 28 (State typing; `helper_preserves`, `run_preserves`, `compute_preserves`, `linear_compute_preserves` in `Step.TypingStates`)**.** *A configuration at a dependent frame is typed when the label of every present occurrence is a judged term there. Every helper firing preserves typing, and so does every run of the helper step. Computation firings that take an occurrence and make one labelled by a computation step of its label preserve typing under the declaration-based preservation hypotheses, and with no hypotheses over the linearized rules of the certified transform program.*

Theorem 29 (Checking as firings; `completed_run_derives`, `Kernel.checking_run_sound` in `Step.TypingChecking`; `identity_checked`, `ground_not_function` in `Step.TypingCheckingControls`)**.** *Goal firings take one goal, make its premise goals, ask guards, tell side conditions as obligations, and carry a local soundness certificate. A run from the root goal alone that ends with no goal present and solvable knowledge derives the root. With the kernel's rules for variables, application, abstraction and the subtype test as firings, a completed checking run gives a typing derivation, given the declaration facts, a formed context and a formed type. Three admitted firings check $\lambda x.x \Leftarrow g \to g$; checking $\lambda x.x \Leftarrow g$ is refused because the told reduction of g to a Π-type has no solution.*

These goal firings have no metavariables yet; postponement and completeness for fair runs are open.

11 Identity and observation

A fresh name is bound: nothing outside the state can tell which fresh name was chosen. A name stops being private when it escapes: a client holds a reference to it, a predicate query sighted it, it was recorded as evidence, or a published row names it. Observation is therefore relative to what observers recorded.

11.1 Private renaming and escape

Theorem 30 (`privatelyEquivalent_iff` in `Step.ObservationViews`; `Client.watch_escaped`, `PrivatelyEquivalent.firing`, `Client.respects`, `ObservationControls.forget_all_not_`

`respected` in `Step.Observation`). *Every identity that a watched run's predicate queries sighted has escaped. Two configurations are privately equivalent for an escaped set E when a renaming that keeps labels and fixes E carries one to a configuration with the same solutions as the other; this holds exactly when their knowledge has the same solutions, the same escaped names are present and the same bag of labels is present. Every firing has a renamed counterpart, admitted together and with equivalent results, and every client's probes respect the equivalence. Forgetting every identity is this equivalence for a client that has seen nothing, and only for such a client: a client holding one occurrence separates states that the empty client identifies.*

The two readouts of fresh identity are the extreme cases: the nominal end, which ignores fresh identities (`nominalEnd_rename` in `AnnotatedRunsQuantum`), is the empty escaped set (`privatelyEquivalent_empty_iff` in `Step.ObservationViews`), and the identity end is the case where every present name has escaped. Two superposed alternatives that make rows of one label under different identities interfere exactly when neither row has escaped, reading 0 while both are private and 2 once one has (`Superposed.nominal_interferes` in `AnnotatedRunsQuantumAggregation`; `Controls.superposed_interferes_iff`, `Controls.superposed_private`, `Controls.superposed_escaped` in `Step.ObservationCoherence`). On frames with an admitted conversion, a readout must give equal results on convertible data; reading the head of a stored redex fails that test (`AdmittedControls.head_not_readout` in `Step.ObservationFrames`).

11.2 Rows modulo the knowledge

Comparing bags of syntactic labels is not fully abstract, in two ways. A query whose region is an arbitrary function of the raw label separates the rows $20 : X$ and $21 : Y$ under the knowledge $X = Y$ (`ModuloControls.syntactic_region_separates`). And the run layer's probes cannot tell the rows X, Y, X from X, Y, Y under empty knowledge, although at the identity substitution one state has two X rows and the other one: a count is a single number that must hold at every solution (`ModuloControls.existing_probes_agree`, `uniform_count_iff` in `Step.ObservationModuloControls`). The repair has two parts: regions read labels through the solution, and a client may *tally* a region, obtaining its count as a function of the solution.

Theorem 31 (Full abstraction; `pointwiseEquivalent_iff_observes`, `moduloEquivalent_iff_observes_of_principal`, `moduloEquivalent_iff_pointwise` in `Step.ObservationModulo`; `firstOrder_moduloEquivalent_iff_observes` in `Step.ObservationModuloControls`). *Two configurations are* pointwise equivalent *when they have the same solutions and the same escaped rows present and, at every solution, some renaming that fixes the escaped rows matches their rows; they are* equivalent modulo the knowledge *when one such renaming matches labels at every solution. Pointwise equivalence holds exactly when every probe that names only escaped identities agrees, where probes are held references, equation guards and absence and count queries at every solution, asks over a region, and tallies of a region; completeness needs only held references, equation guards and tallies. Under a principal answer the two equivalences coincide, so equivalence modulo the knowledge is fully abstract there, and in the first-order profile it is fully abstract for consistent knowledge.*

Without a principal answer the uniform equivalence is strictly finer (`NonPrincipal.pointwise_not_modulo` in `Step.ObservationModuloControls`), and uniform tallies are not fully abstract (`ModuloControls.uniform_not_abstract`). Under $X = Y$ the rows $20 : X$ and $21 : Y$ are equivalent while nothing has escaped and not once one has (`ModuloControls.c2_equivalent`, `ModuloControls.c2_escaped_not_equivalent`).

Theorem 32 (Congruence; `ModuloEquivalent.firing`, `ModuloEquivalent.bisimulation` in `Step.ObservationModuloFirings`). *A firing whose query regions read labels through solutions, and whose made identities are absent from both of two equivalent states, is admitted at one*

exactly when its renaming is admitted at the other, with equivalent results. With an unlimited supply of fresh identities per label, equivalence modulo the knowledge is a bisimulation for such firings.

Each hypothesis is needed: a syntactic query makes admission itself separate equivalent states (`ModuloFiringControls.syntactic_query_collision`), and made identities that collide with present ones leave no counterpart (`ModuloFiringControls.made_collision`, both in `Step.ObservationModuloFirings`). Ordinary query regions therefore read labels through solutions; syntactic observation of labels belongs to reflection on quoted data. Rows made under $X = Y$ interfere modulo the knowledge, reading 0, and report a record with syntactic labels, reading 2 (`ModuloInterference.labelled_rows` in `Step.ObservationModuloCoherence`).

11.3 Observer records and escape-relative transport

Definition 11 (`Record`, `Record.automorphisms` in `Step.Observer`; `Escaped.record`, `EscapeIso`, `Transport.escaping` in `Step.EscapeTransport`)**.** *An observer's* record *is a list of relations among names: held references, sets of names seen together, ordered tuples and recorded choices, each position its own handle. Renaming acts on records as a group, and the automorphisms of a record are its stabilizer. A state of the escape layer carries its public terms and its record; an* escape isomorphism *is an isomorphism of states whose renaming carries one record onto the other and whose frame map carries the public terms up to the knowledge. The escape layer over a natural transport is a transport, and its isomorphisms of a state to itself are exactly the stabilizer of its record (`escapeIso_self_mem` in `Step.ObserverChoices`).*

Theorem 33 (`record_last`, `Escape.traceCategory`, `traceEquiv_record` in `Step.EscapeTransport`; `traceEquiv_correspondence` in `Step.ObserverChoices`)**.** *Records only grow along a run. The escape layer has its own category of dependent traces; trace-equivalent runs end with records related by the closing renaming, and that renaming restricts to a correspondence between the names the two runs exported, which keeps its old part and grows with later exports.*

Firings valued X and Y at a frame of two parameterless holes are one trace while both holes are private and two once X is public (`EscapeControls.private_identified`, `EscapeControls.public_not_identified` in `Step.EscapeHelper`); making occurrence 1 and making occurrence 2 are one trace while private and two once a client holds 1 (`EscapeControls.private_made_identified`, `EscapeControls.escaped_made_not_identified` in `Step.EscapeHelper`). A renaming is invisible to an observer exactly when it maps the observer's whole record to itself: swapping 5 and 6 keeps the sighted set $\{5, 6\}$, so taking 5 and taking 6 after sighting $\{5, 6\}$ are one trace (`ObserverControls.sighting_dissolved` in `Step.ObserverFirstOrder`), and they stay apart when the observer also holds 5, saw the ordered pair $(5, 6)$, or saw $\{5\}$ alone (`ObserverControls.held_distinguishes`, `ObserverControls.pair_distinguishes`, `ObserverControls.singleton_distinguishes`, in `Step.ObserverFirstOrder`). Observers inside the system are part of the state and are renamed with it; only what is revealed to the environment is fixed (`InternalControls.internal_renamed`, `InternalControls.external_blocks` in `Step.ObserverInternal`). Renaming internal records jointly with the state and renaming the state by the automorphisms of a fixed record are one action: pairs over the orbit of a record, up to joint renaming, are equivalent to states up to its stabilizer (`orbitQuotientEquiv` in `Step.ObserverOrbits`). Traces refine observation: in the first-order instance, trace-equivalent runs end in states equivalent modulo the knowledge (`traceEquiv_observes` in `Step.EscapeObservation`; `renaming_traceEquiv_observes` in `Step.ObserverFirstOrder`). The converse is open.

Probes shrink with the record. An observer may use only probes that the automorphisms of its record leave unchanged. Naming only escaped identities is not enough: that permission admits a presence probe that breaks respect (`permitted_present_breaks_respect` in `Step.ObserverChoices`). For records of references, the supported probes are exactly those that name references (`supported_references_iff` in `Step.ObserverRespect`); for sighted sets, equivalence is fully abstract once probes may also count inside the recorded sets, and not before (`recordEquivalent_sightings_iff` in `Step.ObserverSightings`, `supported_not_abstract` in `Step.ObserverAbstraction`); for genuine relations, such as a directed four-cycle, even that is not enough (`relation_not_abstract` in `Step.ObserverSightings`).

Local and global matchings. The renamings that match labels at a set of solutions form a presheaf on sets of solutions (`matching_restrict` in `Step.ObserverSections`). Pointwise equivalence is a non-empty matching at every solution separately, equivalence modulo the knowledge is one renaming that matches at every solution, and a principal answer, being an initial solution, turns a matching there into a global one (`pointwiseEquivalent_iff_stalks`, `moduloEquivalent_iff_global`, `global_of_principal`). Without a principal answer, non-empty local matchings need not have a common element (`NonPrincipal.local_sections_do_not_glue` in `Step.ObserverSections`). The gap is the order of quantifiers over the solutions, local against global. It is not contextuality in the sense of compatible local sections on overlapping measurement contexts that fail to glue [3]: the two local renamings of the control are distinct, so they are not compatible even on the empty overlap. Per-solution renaming cannot move one firing to one firing (`pointwise_not_congruence` in `Step.ObserverPointwise`); a family of firings covers the solutions instead (`counterparts_cover`).

11.4 Symmetry

Theorem 34 (`AnnotatedRuns.no_equivariant_selection`, `no_selection_from_pair`, `ChoiceControls.choice_recorded_identified` in `Step.ObserverChoices`; `no_deterministic_equivariant_commit`, `commitsWithRecord_equivariant`, `commitsWithRecord_breaks_symmetry` in `Step.RaceCommit`; `Twins.uniform_commit` in `Step.RaceProbability`)**.** *A selection that is a function of the observer's record and commutes with every renaming selects neither of two names that an automorphism of the record exchanges; lifted to firings, no deterministic equivariant rule resolves a race between two firings that such an automorphism exchanges. The scope is exactly this: the theorem excludes* deterministic *equivariant selection only. A selection with extra state or an asymmetry may select. A uniform random choice can be equivariant in distribution while each of its outcomes breaks the symmetry. Recording the outcome is a separate contract, not a corollary: the record-carrying resolution, a relation from a record to outcomes each of which extends the record by what the winner took, is equivariant, and each of its outcomes breaks the symmetry.*

11.5 Races and the linear, branching and causal spectrum

A *program* offers firings under labels (rules, rules with sites, or the names of a finite alphabet); a configuration moves under a label when an offered firing is admitted there, and the next configuration is its result. Everything in this subsection is in the first-order one-frame step in `Step.RaceSpectrum` and the modules named below.

Theorem 35 (The race pair; `traceEquivalent_P_Q`, `collect_P_eq_collect_Q`, `not_bisimilar_P_Q`, `traces_and_bags_do_not_decide` in `Step.RacePair`; `bisimilar_iff_sat` in `Step.RaceSpectrum`)**.** *Let rule A take a and go and make k, and let rules B and C each take k. From P, A fires and leaves B and C racing for its k; from Q, two firings of A race for a, after which only B or only C can fire. P and Q have the same traces and the same*

collected bags of answers at every budget of the step's bounded search, and they are not bisimilar: $\langle A\rangle(\langle B\rangle\top \wedge \langle C\rangle\top)$ separates them. For image-finite programs, bisimilarity is agreement on Hennessy–Milner formulas [30]. Equal traces and equal answer bags at every budget therefore do not determine the bisimulation class.

Theorem 36 (The spectrum; `bisimilar_iff_traceEquivalent`, `Fragment.bisimilar_iff_traceEquivalent` in `Step.RaceDeterminate`; `bisimilar_of_stepBisimilar`, `staticIndependent_iff_provenance`, `stepBisimilar_of_hpBisimilar` in `Step.RaceSteps`; `step_strictly_finer`, `hp_strictly_finer` in `Step.RaceStepControls`)**.** *Where same-labelled successors are bisimilar, bisimilarity is trace equivalence [42]; the fragment of ground firings with one label each and pairwise statically independent offers is of this kind, and the race pair lies outside it. A* step move *fires a nonempty word of pairwise statically independent firings, labelled by the multiset of their labels; step bisimilarity implies interleaving bisimilarity and is strictly finer ($a \mid b$ against $a.b + b.a$), so the expansion law fails. On pure firings without reuse, independence is provenance: neither firing takes what the other made. History-preserving bisimilarity [60] implies step bisimilarity and is strictly finer ($a \mid b$ against $(a \mid b) + a.b$).*

Theorem 37 (Readouts and records; `readout_eq_of_openBisimilar`, `Triggered.experimenter_separates`, `Triggered.not_congruence` in `Step.RaceExperiments`; `Copying.readout_strictly_coarser` in `Step.RaceCopying`; `records_blind_to_causality`, `provenance_record_determines_past` in `Step.RaceRecords`; `choiceEscape_records`, `commit_timing` in `Step.RaceCommit`; `Die.not_probBisimilar` in `Step.RaceProbability`)**.** *An experimenter that adds atoms at the ports of an open system and watches which labelled firing occurs, or quiescence, is sound for open bisimilarity and separates a triggered race pair; closed bisimilarity is not a congruence for such interventions. Without copying a state, such readouts are strictly coarser than bisimilarity ($a.(b.c + b.d)$ against $a.b.c + a.b.d$), as testing characterizations of bisimulation lead one to expect [2]. For firings without predicate queries the escape's record is what was taken; records do not carry causality, and recording what was made as well would determine the causal past. When a race was resolved is invisible in the answer bag and visible to history-preserving identity. With weighted firings, the die pair has equal trace distributions for every weight and is not probabilistically bisimilar [32] for weights strictly between* 0 *and* 1*.*

Equivalence modulo the knowledge is strictly finer than rule-labelled bisimilarity (`moduloEquivalent_strictly_finer` in `Step.RaceRecords`). The weaknesses of this layer are listed with the open items in Section 19.

12 Models

Theorem 38 (An independent set model; `setModel`, `firstOrder_model_transition`, `fire_val_map`, `firstOrder_realized`, `run_conservation` in `Step.ModelFirstOrder`; `AnnotatedRuns.firstOrder_set_model` in `Step.ModelConsistency`)**.** *Every first-order structure, in any universe, is a Henkin model of the one-frame first-order step: points are assignments and evaluation is the structure's. At every point that satisfies an admitted firing's result, the earlier knowledge holds, and the taken, read and made terms evaluate to the taken, read and made values, so the present values lose what was taken and gain what was made. With a nonempty carrier, every admitted firing has a point of its information, and along a run the end values plus everything taken equal the start values plus everything made, at every point; that flow is a site account.*

Theorem 39 (Contextual set models; `contextualSetModel`, `eval_instantiate`, `contextual_realized` in `Step.ModelContextual`)**.** *A set algebra of a binding signature reads an argument that binds variables as a function of their values. Contextual instantiation is evaluation at the pulled-back point, and with inhabited sorts every admitted firing of every step over contextual frames, the helper step and the native instance included, has a set point of its information.*

Theorem 40 (Proof logic; `CertainChecks.Derives`, `derives_iff_accepts`, `derives_iff_entails`, `derives_sound_free`, `derives_iff_termModel`, `derives_herbrand_sound`, `admitsDerivably_iff` in `Step.ModelDerivation`)**.** *The equation guards and the absence and count queries of the first-order profile have a calculus of three rules: equality at the unifier's answer, absence by refuting each candidate, and count by both; it equals the existing executable procedure for them. A guard or query is derivable exactly when the store is consistent and it holds at every solution. A derived one holds at every satisfying point of every free model in every universe, and one set model, the free term algebra, is sound and complete for the calculus. The ground-term model is sound and not complete: with one constant c, $X = c$ holds at every ground point and is not derivable (`oneConstant_separates`). For query-free firings, admission is a derivable judgment.*

Theorem 41 (Consistency; `certainChecks_consistent` in `Step.ModelConsistency`)**.** *For distinct constants $c \neq d$, the equation $c = d$ is derivable from no store, and no firing that tells $c = d$ is derivably admitted. The proof is by soundness into the free term algebra; it needs no universe, no inaccessible cardinal and no higher-order logic.*

That such a firing is refused (`tellClash_refused` in `Step.ModelConsistency`) is a separate, operational fact. Guard formulas with $\top$, $\bot$, equations, absence, count and the connectives are read at solutions through the solution's kernel (`GuardFormula`, `satisfies_congr` in `Step.ModelGuards`); query-free first-order firings lie in the fragment where admission is shapes, enabling and two asks over the solutions (`Step.admits_iff_asks`, `firstOrder_valid_iff_asks`).

Theorem 42 (Presheaf and Henkin semantics; `entails_iff_le_interior`, `principal_iff_generated`, `entails_iff_forces` in `Step.ModelPresheaf`; `HenkinModel`, `entails_iff_free`, `solvable_iff_generic`, `closedEvery_iff_stage` in `Step.ModelHenkin`; `closedAdmits_iff_stage`, `HenkinModel.no_point_of_unsolvable` in `Step.ModelConsistency`)**.** *Knowledge denotes a cosieve of solutions, a guard the persistent interior of where it holds, and an ask is the inclusion of one in the other. An answer is principal exactly when it generates the cosieve of solutions. Asks coincide with Kripke forcing, with Cohen's clause for negation and no covers, on the flat fragment of guard formulas. A Henkin model [29] of the frames is a presheaf of points with a natural evaluation that respects the frame equivalence; free models, whose points are realized by syntactic solutions, are a sound and complete class for asks; a store is solvable exactly when it has a generic point; closed asks and closed admission are validity in the closed stage; and a free model realizes no unsolvable store.*

Controls in `Step.ModelControls`: the case split $X \in \{\mathsf{true}, \mathsf{false}\}$ is not asked, holds at every ground point, and is forced on each branch; excluded middle is asked and not forced. If f is constant in a set model, $fX = fY$ holds there while $X = Y$ does not, although the syntax asks it, so plain set models are not sound for asks (`constantModel_not_free`); set models also satisfy $X = fX$, which the occurs check refuses. A point here is an environment of a frame; no claim is made that set models are points of a topos. The case-split cover would force $X = \mathsf{true} \vee X = \mathsf{false}$, which is not asked, so the semantics uses no covers.

13 Calculi as firings

13.1 Resolution

Theorem 43 (`resolve_sld_step`, `resolve_commits`, `resolve_refused_iff` in `Step.ResolutionFiring`; `Controls.split_leaves_residue`, `Controls.atomic_refused` in `Step.ResolutionControls`)**.** *A resolution firing takes a goal, makes the renamed subgoals of a clause and tells the equation between the goal and the renamed head. With θ the occurs-checked unifier's answer for the knowledge and a renaming fresh for it, the firing is admitted exactly when θ of the goal unifies with the renamed head, and with a most general unifier μ the result*

has the principal answer $\mu \circ \theta$: one firing is one SLD step [50, 34]. An admitted firing replaces the goal by all the subgoals, keeps everything else and adds exactly one equation; a refused one has a stated reason and starts no run. For the goal $r(X, f(X))$ and the clause $r(Y, Y) \leftarrow q, q$, a realization split into a take and a later tell is admitted halfway and then stuck with the goal consumed and one subgoal made, while the atomic firing is refused with no residue.

Theorem 44 (Answers; `Holds.least`, `sound`, `complete`, `resolve_independent_iff`, `resolve_switch_traceEquiv`, `traceEquiv_same_answers` in `Step.ResolutionAnswers`)**.** *The declarative semantics is the least set of atoms closed under clause instances; for ground atoms it is the least Herbrand model. Every run finds only true answers, with no freshness needed. If a solution of the knowledge makes every present goal true, then for every selection rule there is a run that follows the rule, resolves every goal and ends with a solution that agrees with the given one on the configuration's variables; the choice of goal is the choice of firing, so no fairness in choosing goals is needed. Two resolution firings are independent exactly when they resolve different goals and their unifications are jointly consistent; swapped runs are trace equivalent through the step's own swap, and trace-equivalent runs have the same answers.*

Theorem 45 (Variant tabling; `variantStateIso`, `VariantEquivalent.answers`, `PrivatelyEquivalent.variant` in `Step.ResolutionTabling`; `ModuloVariant.answers` in `Step.ResolutionModulo`)**.** *A permutation of variables is an automorphism of the one frame and, with a covering occurrence renaming, an isomorphism of states. Calls that are variants have the same answer instances, so one table serves both [56], and tabling modulo the knowledge is sound in the same way. Private equivalence alone renames occurrence identities, not variables, and cannot key a variant table.*

Controls in `Step.ResolutionControls`: $p(X)$ and $p(Y)$ share answer instances and are not privately equivalent (`Controls.variants_not_private`); $p(a)$ is not a variant of $p(X)$ (`Controls.instance_not_variant`); $p(X)$ under $X = a$ and $p(a)$ have the same answers and are not variants (`Controls.modulo_not_variant`); two clauses for one goal are both admitted and not independent (`Controls.two_clauses_dependent`).

Clauses stored in the space. In the read form a clause is a stored row that a firing reads, not takes; each read opens fresh copies of the clause's variables at a scope outside the occupied names, so two uses never interfere and a goal's variables are never captured (`ReadPort.resolve_valid_iff`, `ReadPort.resolve_clause_stays`, `ReadPort.two_reads_apart` in `Step.ResolutionReadPort`). The read form and the one-frame firing on the opened clause have the same solutions once the clause's own variables are forgotten (`ReadPort.solvable_iff_builtIn` in `Step.ResolutionReadPort`). Without renaming apart, $p(X)$ against an unrenamed $p(f(X))$ tells $X = f(X)$ and is refused; the read form admits it (`Controls.capture_refused` in `Step.ResolutionControls`; `ReadPort.Controls.capture_free_admitted` in `Step.ResolutionReadPortControls`). The run-level theorems above are proved for the one-frame form over first-order terms, which the read form reduces to firing by firing; carrying them to runs of the read form over growing frames is open.

13.2 Superposition

Theorem 46 (Ground inferences; `clauseLT_wf`, `Inference.sound`, `Inference.concl_lt` in `Step.SuperpositionGround`; `Lit.lt_iff_multiset` in `Step.SuperpositionLiterals`; `inferFiring_admits_iff`, `simplify_takes_redundant` in `Step.SuperpositionSaturation`)**.** *Over a linear, well-founded order on ground terms that is compatible with contexts and contains the strict subterm relation, literals are ordered as in Bachmair and Ganzinger [6] and clauses by the multiset extension [19], which is well founded and total. Positive and negative superposition, equality resolution and equality factoring are sound and reductive. A ground inference firing reads*

its premises and makes its conclusion; it is admitted exactly when the premises are present, the made occurrence is fresh and carries the conclusion, and the clauses form an inference. Demodulation, subsumption, tautology deletion and the deletion of a literal $s \not\approx s$ take only clauses that are redundant with respect to the remaining ones.

Theorem 47 (Completeness and the given-clause loop; `candidate_models`, `empty_mem_of_unsatisfiable` in `Step.SuperpositionCompleteness`; `Derivation.saturated`, `Derivation.complete`, `Loop.fair` in `Step.SuperpositionDerivation`; `givenClause_complete`, `givenClause_sound` in `Step.SuperpositionGenerator`). *A saturated set of ground clauses without the empty clause has a model, the candidate model built by well-founded recursion; so an unsatisfiable saturated set contains the empty clause. Fair derivations reach saturation and derive the empty clause from an unsatisfiable start, and every derived clause follows from the start. The given-clause loop that selects the oldest clause is fair, with a generator proved to list every inference, and it is sound and complete.*

The loop's outcomes read in the outcome algebra of Section 14: the empty clause found means unsatisfiable; saturation without it means satisfiable, a reading that is sound only by the completeness theorem; a cut-off run claims nothing (`readout_cutOff` in `Step.SuperpositionDerivation`). A loop that never selects a needed clause misses the refutation (`Controls.fair_refutes`, `Controls.starving_never_refutes` in `Step.SuperpositionControls`); in that control the starving policy never selects the clause by construction.

Theorem 48 (Non-ground firings on the shared store; `unifyFiring_admits_iff`, `unifyFiring_principal`, `sup_admits_iff`, `eqRes_admits_iff`, `eqFact_admits_iff`, `resolution_admits_iff` in `Step.SuperpositionFiring`; `admits_lift_iff`, `guarded_closes`, `guarded_independent_iff`, `entails_closure_iff` in `Step.SuperpositionGuarded`). *A non-ground inference firing reads its stored premises, tells the unification between renamed-apart copies, makes the encoded conclusion and guards the closure of the side condition. At a principal answer θ it is admitted exactly when its premises are present, its made occurrence is fresh and carries the conclusion, and some most general unifier μ satisfies the side condition; its result has the principal answer $\mu \circ \theta$. Binary resolution is a special case, and resolution firings embed unchanged, so both calculi run on one store. Ordering side conditions survive generalization, not instantiation; their closure (some more general solution satisfies the condition) is stable, and asked at every solution it equals the condition at the principal answer. Diamonds close, and independence is the four admissions.*

Two admitted inferences with different made occurrences are independent and their two orders are trace equivalent; a simplification that takes another inference's premise makes the two dependent (`infer_independent`, `infer_traceEquiv`, `simplify_takes_premise_dependent` in `Step.SuperpositionSaturation`). Asking a co-stable guard at every solution is too strong, and its closure is admitted (`ControlsFiring.notAbove_every_refused`, `ControlsFiring.notAbove_closure_admitted` in `Step.SuperpositionControlsFiring`). The completeness of λ-superposition [9] rests on its own unifier, and the step records how a firing reads it: a bounded unifier enumeration that is cut off never reads "no unifier", and over a minimal complete set of unifiers the step's single firing is admitted exactly when the side condition holds at every member, the readings coinciding in the pattern fragment (`UnifierEnumeration.not_refuted_of_cutOff`, `readings_complete`, `readings_principal` in `Step.SuperpositionHigherOrder`).

Theorem 49 (Lifting to non-ground clauses; `lift_binary`, `overlap_redundant`, `SupLift.admitted`, `NSaturated.satisfiable`, `NDerivation.complete` in `Step.SuperpositionLifting*`; `KBOParams.grounded` in `Step.SuperpositionKBO`). *Let the non-ground ordering be stable and linked to a ground simplification ordering: $s \succ t$ implies $s\theta > t\theta$ for every grounding θ. The Knuth–Bendix ordering is such an ordering.*

1. *Every ground inference among instances of stored clauses is either the instance of a non-ground inference firing, admitted on the shared store at a most general unifier through which the grounding factors, or an inference inside the value of a variable, which is redundant.*
2. *The firing's guard is the closure of the side condition. The conditions hold at the unifier because they hold at the ground instance and survive generalization; the bare condition asked at every solution would refuse the firing.*
3. *A non-ground set saturated up to redundancy and without the empty clause has a model of all its ground instances; a fair non-ground derivation from an unsatisfiable start derives the empty clause.*

A further admission theorem, `unifyFiring_admits_of_consistent`, admits inferences on derived clauses, whose variables the knowledge already binds, without renaming apart.

Open: the non-ground given-clause loop on the shared store, with its generator and fairness; a check that a firing's literal parameters are a renamed copy of the premise it reads; the full multiset extension of the ordering to literals. Sharing one step proves nothing about the completeness of a mixture of calculi: each calculus keeps its own inferences, redundancy and fairness.

14 Search

A bounded search returns a multiset of answers and a status: finished, or cut off by its budget.

Theorem 50 (The outcome algebra; `Outcome.choice_comm`, `Outcome.choice_assoc`, `Outcome.bind_assoc`, `Outcome.add_bind`, `Outcome.bind_add`, `Outcome.unknown_bind`, `Outcome.collect_unique`, `Outcome.verdict_ne_refuted_of_cutOff`, `Outcome.verdict_eq_of_le` in `Step.SearchOutcome`)**.** *Choice adds answer multisets and joins statuses: a commutative monoid whose unit is the refuted outcome, finished with no answers. Bind is a monad that distributes over choice on both sides, with the refuted outcome a left zero and the unknown outcome, cut off with no answers, left-absorbing. Collecting all answers into one value is possible only once a search is finished: on an inhabited type, sending cut-off outcomes to unknown is the only monotone map that collects finished outcomes. An outcome reads as one of three verdicts:* answered, refuted *(finished with no answers) or* suspended *(cut off with no answers). Incomplete is never none: a cut-off outcome never reads refuted, and definite verdicts are stable as the budget grows.*

The laws that fail are part of the algebra: there is no right zero, since a cut-off search whose answers all fail later stays cut off (`Outcome.cutOff_bind_refuted_ne_refuted`); choice is not idempotent; collecting is not additive; and on lists, choice is not commutative (`Outcome.list_choice_not_comm`). Answers form a multiset because alternatives are unordered and two derivations of one answer are two answers.

Theorem 51 (Budgets and policies; `mem_run`, `run_eq_of_finished`, `status_run_eq_cutOff_iff`, `beyond_cut`, `refuted_sound` in `Step.SearchSpace`; `covers_search`, `within_refuted_sound`, `take_finished_contract`, `bfs_complete`, `deepen_complete` in `Step.SearchPolicy`; `within_verdict_ne_refuted`, `all_dom_iff`, `collect_eq_all` in `Step.SearchFinish`)**.** *For a space that unfolds each state into answers and states, the run at budget n reports exactly the answer addresses of length at most n, each once; a finished run is stable; a run is cut off exactly when a state lies at depth at least n, and every answer below such a state is real and unreported, so a refuted readout is sound for the unbounded semantics. A frontier machine covers each address exactly once under every policy. Of the search combinators,* `within`*'s refutations are sound and its cut-off results never read refuted;* `take` *finishes with its quota or with the whole search; and* `all` *is defined exactly when the tree is finite, is then the same under every policy, and equals the collected finished run. Breadth-first search and iterative deepening are complete.*

Depth-first search is not fair: in a left-recursive space it is cut off with {0} from budget 2 on and never finds the answer 1, which breadth-first search finds at budget 5 and iterative deepening at depth 2, and no policy finishes that space (`Controls.dfs_loses`, `Controls.bfs_finds`, `Controls.deepen_finds`, `Controls.never_finishes` in `Step.SearchControls`). At zero budget the outcome is suspended, not refuted, although answers exist (`Controls.zero_budget`).

Theorem 52 (Runs of the step as a search space; `mem_run_space`, `collect_space`, `checkOfSearch`, `checkOfSearch_unknown_iff`, `staged_suspended_not_refusal` in `Step.SearchStep`; `Representatives.complete`, `exists_similar` in `Step.SearchClasses`)**.** *With finitely many admitted firings per state, the runs of the step form a search space whose answers are complete runs: budget n reports the complete runs shorter than n by their final states, and collecting gives those states when the search is finished and unknown when it is cut off. Where a state admits infinitely many alike firings, finitely many representatives up to alikeness suffice: every run is similar to a run of representatives, and the space is sound and complete up to state classes. A search for independence certificates is a sound three-valued check at every budget, and it answers unknown exactly when the search is suspended. Reading a suspended deferred-effects block as a refusal contradicts the source contract of an executable block with a visible effect.*

Search outcome	Existence	Independence check	Deferred-effects block
some answer, any status	answered	independent	publishes the composite
finished, no answer	refuted	dependent	caller unchanged, non-admission proved
cut off, no answer	suspended	unknown	caller unchanged, nothing claimed

Memo tables are sound only when every stored outcome keeps its status: storing a run with its own status is sound, a state whose search never finishes has no sound finished record, and a cut-off record misread as finished refutes a state that has answers (`soundMemo_approx`, `finished_record_unsound` in `Step.SearchTabling`; `Controls.misread_refutes` in `Step.SearchControls`). The search combinators follow the tradition of search combinators for constraint programming [53].

15 Rational trees and the occurs-check policy for types

Theorem 53 (Rational frames; `rationalFrames`, `eq_of_bisim`, `rational_iff_unfold`, `unfold_eq_iff`, `solves_unique` in `Step.RationalTrees`)**.** *Possibly infinite trees with finitely many distinct subtrees form a substitution action with equality as its equivalence, hence a one-frame instance. Equality is equality of unfoldings, proved by coinduction; a tree is rational exactly when it unfolds from a node of a finite graph; two graphs present one tree exactly when they are bisimilar; and guarded systems have unique solutions.*

The frames carry trees, not graphs, because grafting graphs is associative only up to bisimilarity while the interface asks its action laws as equalities.

Theorem 54 (Unitary unification; `principal_or_refuted`, `consistent_iff_not_refuted`, `mgu_factors` in `Step.RationalUnification`; `unfold_eq_iff_agree`, `consistent_rstore_iff_unifiable` in `Step.RationalDecision`; `solvedSubst_principal` in `Step.RationalSolvedForm`)**.** *Every finite store has a most general idempotent solution or a clash of two rigid labels, never both, and a clash is the only refutation: $X = f(X)$ is solved by $f(f(f(\cdots)))$, and together with $Y = f(Y)$ it forces $X = Y$. On finite presentations equality is decided by agreement to a bounded depth, unifiability by a node closure within a bounded number of iterations, and the solved form is computed as a finite graph.*

Theorem 55 (The finite fibre; `models_ofStore_iff`, `consistent_ofStore`, `consistent_iff_acyclic` in `Step.RationalFibre`; `occurs_not_reflected` in `Step.RationalControls`)**.** *Finite*

solutions are rational solutions and finite most general solutions stay most general. A finite problem is finitely solvable exactly when its rational solved form is finite, and $X = f(X)$ shows that solvability does not reflect in general.

The tree model is a free Henkin model in which asks are validity, and a three-rule calculus over the solved form, with no occurs check, is sound and complete for it and consistent (`treeModel`, `entails_iff_treeModel`, `RationalChecks.derives_iff_treeModel` in `Step.RationalModels`, `rationalChecks_consistent` in `Step.RationalAdmission`). The step over rational trees has a lawful transport (`rationalStep`, `rationalLawful` in `Step.RationalStep`), and the firing that tells $X = f(X)$ is admitted there from no knowledge and refused over finite trees at every configuration (`Controls.tellLoop_admitted`, `Controls.tellLoopFinite_refused` in `Step.RationalStep`). The two profiles are siblings, not a relaxation of one another: the finite calculus derives that no occurrence labelled X matches $f(X)$, by the occurs check, which is false over rational trees and not derivable there (`Controls.absence_not_transferred` in `Step.RationalControls`). Rational trees are immutable data: a shared node and two copies unfold alike, and distinct variables stay distinct (`Raw.sharing_unobservable`, `Raw.distinct_variables_unshared` in `Step.RationalTrees`).

Types keep the occurs check. Over finite types $\lambda x.\, x\, x$ is untypable, while over rational types Ω is typable at $T = T \to T$ and has no normal form (`Controls.selfApplication_untypable`, `Controls.omega_typable`, `Controls.omega_no_normal_form`, `Controls.cyclic_type_breaks_normalization` in `Step.RationalTypeControl`). The rational profile is therefore stated for object terms only, and the dependent fibre keeps its occurs check.

16 Charges, folding and the affine pump

Theorem 56 (Conserved charges; `ConservedCharge`, `run_balance`, `not_fold_of_unbalanced`, `traceEquiv_net`, `intCharge_wrap`, `weightCharge`, `weightCharge_invariant` in `Step.Charge`; `valueCharge_totals`, `place_invariant` in `Step.ChargeFirstOrder`)**.** *A conserved charge is a value on states with what each firing takes and makes, balanced at every admitted firing; summed along a run the books balance. For a charge invariant under transport with values in a left-cancellative monoid, a run that returns to its start up to isomorphism took exactly what it made (`balanced_of_fold` in `Step.Charge`), so an unbalanced run proves a state new without comparing states; net flux is a property of traces for invariant charges; and an integer charge read modulo 2^{64} remains a sound filter. Occurrence weights give a charge of every step, invariant whenever renamings keep the weights. In the first-order instance the value flow of the set model is such a charge, and weights that every rule of a run balances give a place invariant [44].*

Equal charges never prove two states the same: every place charge agrees on two states that the knowledge $X = 1$ separates (`ChargeControls.charges_blind_to_knowledge` in `Step.ChargeFirstOrder`), and the length of a run is a trace property that no invariant charge measures (`ChargeControls.length_traced_not_conserved`).

Theorem 57 (Canonical interleavings; `Inverted.swap`, `exists_canonical`, `exists_canonical_of_check`, `exists_canonical_same_last` in `Step.Canonical`; `firstOrderKeyLaws`, `firstOrder_exists_canonical` in `Step.CanonicalFirstOrder`)**.** *Give firings a key kept by moves, alikeness and residuals. A run is canonical when no adjacent pair inverts the key order while an oracle declares the pair independent. One swap removes exactly one inversion and keeps trace equivalence; every run is trace equivalent to a canonical one, with any sound three-valued check as the oracle and unknown read as dependent; and every end-state class is reached. In the first-order instance every key satisfies the laws.*

Three independent producers are explored in one order instead of six, a pair that does not commute is kept in every order that runs, and three tells that commute pairwise are never fired

together (CanonicalControls.clique_explored_once, CanonicalControls.dependent_pair_kept, CanonicalControls.hollow_cube_never_round in Step.CanonicalFirstOrder). This is partial-order reduction [26]; the rule is complete and not minimal, since a trace class can contain more than one canonical run.

Theorem 58 (Folding; Canonicalizer, fold_reach, fold_iterate, escape_fold_quiet, record_automorphisms_append in Step.Fold; renaming_key_iff, renaming_fold_observes in Step.FoldFirstOrder)**.** *States with equal keys are matched run for run by similar runs, and a fold of a descendant onto its ancestor replays the loop any number of times. In the escape-relative transport, a run that ends in its start's class recorded nothing, and every closing isomorphism renames by an automorphism of the start's record; disclosure shrinks the folding group, since the automorphisms of a concatenated record are the intersection of the two groups. At a fixed record, two states share a key exactly when a label-keeping renaming in the record's automorphism group carries one to a configuration with the same solutions as the other, and folding under that relation keeps every supported probe modulo the knowledge.*

A swap of a sighted set folds; a held reference blocks it; and canonicalizing each sighted set on its own is wrong when the sets overlap (FoldControls.sighting_swap_folds, FoldControls.reference_blocks_fold, FoldControls.overlap_blocks_fold in Step.FoldFirstOrder). The key is specified, not computed; computing it is canonical labelling.

Theorem 59 (The affine pump; affine_pump in Step.FoldPump)**.** *Suppose a word of query-free firings runs from a configuration C and ends in $C \otimes \delta$ up to equivalence, where δ carries no knowledge and lies beside C, and suppose that for every finite set of identities a label-keeping renaming fixes the occurrences of C and moves what the word makes off that set. Then for every n an executable word ends in $C \otimes E$, where E carries no knowledge, lies beside C and has n times the labels of δ.*

A producer from the empty configuration pumps to n occurrences for every n, while a rule that makes b and requires b absent fires once and is then refused under every renaming (PumpControls.producer_pumps, PumpControls.absent_guard_stops in Step.FoldPump): absence and count queries see what the cycle has made. The proof goes through the run layer's frame rule up to renaming (frame_rule_renamed in AnnotatedRunsSeparationFrame), whose side conditions are that made identities avoid the frame, that the knowledge is compatible and that no query region meets the frame.

A Noether correspondence in the sense of Baez and Fong. A state function commutes with a Markov generator exactly when its mean and its second moment are conserved [5]; for a finite step and a generator whose transitions between distinct states are exactly its firings, a potential commutes with the generator exactly when it is a zero-flux conserved charge (baez_fong, step_noether in Step.ChargeNoether). A permutation symmetry gives no such charge: the swap commutes with $\left(\begin{smallmatrix} -1 & 1 \\ 1 & -1 \end{smallmatrix}\right)$, yet only constant diagonals do (NoetherControls.symmetry_without_charge).

17 Transfinite runs

Two results take the step to infinite objects; their reading is developed in a companion working note [57], and only the statements are recorded here.

Theorem 60 (ω-limits; ConvergesTo.unique, Productive.existsUnique_fixed, tellLoop_unfolding_limit in Step.TransfiniteLimits)**.** *A run of trees converges to a tree when at every depth it is eventually equal to it below that depth; limits are unique, and a productive transform of whole trees has exactly one fixed point, to which the run of its iterates converges from every*

start. The rational step solves $X = f(X)$ in one firing, the unfolding run $X, f(X), f(f(X)), \ldots$ converges to that solution, and it is the only solution. A transform that flips the root between two constants has no convergent run, and the unguarded $X = X$ has more than one solution (`not_convergesTo_rootFlip`, `unguarded_not_unique`).

Theorem 61 (Hyperfinite runs; `HyperRun.balance`, `HyperRun.balance_weight`, `germ_ne_iff`, `HyperRun.eventually_not_fold`, `HyperRun.taken_eq_made_of_eventually_fold`, `HyperRun.const_lt_length`, `cofinite_germ_ne_not_eventually_ne` in `Step.TransfiniteUltrapower`)**.** *A family of runs read modulo an ultrafilter conserves every conserved charge in the ultrapower, occurrence weights included. For a charge invariant under transport with values in a left-cancellative monoid, a hyperfinite run that does not take what it makes has almost every coordinate run failing to fold back to its start; this direction uses that the filter is an ultrafilter, through Łoś's theorem for a negated equation, and over the cofinite filter, which is not one, germs can differ without differing eventually. Conversely, if almost every coordinate run folds, the hyperfinite run takes what it makes. Over a free ultrafilter, if for every n only finitely many coordinate runs have length at most n, the hyperfinite length exceeds every standard number.*

18 Related work

Rewriting logic. A rewrite theory has a signature, equational axioms and rewrite rules; its sequents are concurrent computations, and in its initial model the proof terms, up to the equations of categories and of functorial operators, form a category whose arrows are those computations. The interchange law identifies the serializations of independent rewrites, and parallel composition is functoriality of the operators [38]. The trace category above has the same role for firings that are constrained, guarded and frame changing: swaps of independent firings play the part of interchange. Transport has no counterpart there, because a rewrite theory's terms live over fixed variables. Several later developments meet constructions of this note:

- frozen arguments forbid rewriting below declared positions [11]; dormant schema rows, which are never matched as data, play that role;
- rewriting modulo SMT rewrites symbolic states carrying a constraint and keeps only satisfiable states [51]; a firing's information and its solvability are the same idea over higher-order patterns at frames, with asks quantified over every solution;
- narrowing computes over all instances of a symbolic state [39]; the pattern driver and the three-valued check serve that purpose here;
- a multiset of objects and messages at the top of the state, matched modulo associativity, commutativity and identity, is the state of a rewrite theory for object systems; the space of a state is such a multiset, with identities.

Absence and count, asked at every solution with coverage certificates, are expressed in rewriting logic only through rule conditions.

Constraint rewriting and logical frameworks. Takes, reads, makes and tells are those of constraint handling rules with identified constraints [25], whose linear-logic semantics [10] reads taken constraints linearly and kept ones persistently; ask and tell over a store are those of concurrent constraint programming [52] and its linear variant [21]. Equivalence of states up to renaming of local constraints modulo the store is studied for constraint handling rules in [49]; how it relates to the pointwise and uniform equivalences modulo the knowledge above is open. The concurrent logical framework CLF [15] and its implementation Celf [12] combine dependent types, linear and persistent contexts, fresh names and a concurrent equality of computations; substructural logical specifications [54] make traces objects and type states by generative invariants. CLF's concurrent equality is syntactic, as is the static independence of the run layer; the step's independence is semantic and depends on the state.

Residuals, concurrency and process equivalence. Residuals and permutation equivalence go back to Lévy [33]; concurrent transition systems [55] and axiomatic rewriting theory [37] state the cube and standardization laws abstractly; traces of words are Mazurkiewicz's [36]; event structures [62] and higher-dimensional automata [48, 59] record which sets of events can occur together, not only pairs, as the cells above do. Graph rewriting with negative application conditions [28] is the nearest precedent for absence guards, and contextual nets [43] and their asymmetric event structures [7] for reads that do not consume. The races section places the step on the linear-time/branching-time spectrum [58]: Milner's determinacy theorem [42], the Hennessy–Milner characterization [30], step and history-preserving bisimulation [60], testing with copying [2] and probabilistic bisimulation [32] appear there as theorems about firings. Partial-order reduction [26], place invariants [44] and the Noether theorem for Markov processes [5] are the precedents of the charges section.

Logic programming and saturation. Resolution [50] and the SLD semantics, with its least Herbrand model and the independence of the selection rule, follow Lloyd [34]; tabling follows Tamaki and Sato [56] and SLG resolution [13]. Superposition with redundancy, saturation and candidate models is Bachmair and Ganzinger's [6], over the multiset ordering of Dershowitz and Manna [19]; λ-superposition [9] rests completeness on complete sets of unifiers, as the outcome readings above do.

Infinite trees and non-well-founded sets. Rational trees as the domain of a logic language are Colmerauer's [17]; their fundamental properties are Courcelle's [18], and the complete axiomatizations of finite, rational and infinite trees are Maher's [35]. Unique solutions of guarded systems correspond to the solution lemma of non-well-founded set theory [4, 8].

Names, binders, syntax and dependent types. Fresh and escaped names are those of nominal sets [46] and of the ν-calculus [47]. Declaration families with metavariable arities are the second-order fibre of contextual modal type theory [16]; the pattern fragment is Miller's [40], raising is from unification under a mixed prefix [41], and existence of solutions for flexible pairs is Huet's [31]. Dependent unification and its pattern fragment are Pfenning's and Abel and Pientka's [45, 1]; the dependent frames are built over a category with families [20]. Second-order abstract syntax is the initial algebra of a binding signature [24] with a complete equational logic [23], and it extends to dependently sorted syntax [22]. Native type theory [61] derives a dependent internal language from a presented language through presheaves. Henkin models [29] and Kripke forcing are the precedents of the model section, and the sheaf-theoretic account of contextuality [3] marks what the local-global gap of observation is not.

What is new here. Asks quantified over every solution of a residual, non-ground store, with coverage certificates for absence and count; footprints indexed by solutions; diamonds that close up to isomorphism as frames grow; a sound three-valued check that keeps independence defined where unification is undecidable; escape of names by query sightings; publication recipes fixed before execution; and the combination of these in one step of which typing contracts, resolution, superposition and bounded search are instances.

19 What is proved and what is open

19.1 Proved

Modules	What they prove
`Step.Interface`	frame actions, solvability, asks, principality; the step; runs and site accounts
`Step.Transport`, `Step.PrincipalAnswers`	isomorphisms of frames and of states with renamings; terms carried up to the knowledge; transport up to alikeness; strict laws; state classes; similar runs
`Step.LawfulTransport`, `Step.LawfulFirstOrder`, `Step.LawfulHelper`	local guards, conservative settlement, coherent residuals; independence independent of the lawful residual; first-order and helper instances; controls
`Step.Independence`, `Step.TraceCategory`	independence; swaps; trace equivalence and its congruence; site accounts of traces; deciding by admissions; the three-valued check; the category of dependent traces
`Step.Cells`, `Step.CellsFirstOrder`, `Step.CellsHelper`	cells; 2-cells as independent pairs; first-order cube law; the hollow cube; helper cubes and swap coherence
`Step.FirstOrderIndependence`, `Step.StaticTraces`, `Step.GroundedRegions`	the one-frame transport; closing diamonds; static independence and static traces; the round condition; the overlap control; grounded regions; the first-order decision
`Step.ContextualDiamond`	the helper step; its transport; closing up to renaming; the concrete diamond; the pattern-fibre decision; the unknown control
`Step.Dependent*` (Frames, Helpers, Controls, Closed, ClosedTower, Fibre, Packing, Conversion)	the dependent instance; dependent helpers and the swap criterion; closed asks and the empty-type trap; the non-dependent fibre; transport up to conversion
`Step.Translation*`, `Step.NativeUniverseFrames`	the raising translation; reflection of solutions and asks; admission transfer for definitions; the η defect and its repair; the object-theory comparison; categories with families and the native universe as frames
`ContextualDependentEta*`, `ContextualDependentConstraints`, `ContextualDependentElimination`	the canonical typed η instance; the bridge from the dependent frames; η controls; conversion scans and elimination preserving solutions
`Step.BetaEtaReadings`, `OSLF.Syntax.HereditaryBetaEta*`	canonical forms by hereditary substitution; framework $\beta\eta$ unification; the admitted object theory and its boundaries
`Step.NativeInstance`, `Step.NativePublicationControls`	native firings as firings of the step; recipe-published results; the raw target without recipes; published and raw disagree
`Step.Typing*`	contracts as firings and their adequacy; evidence reuse by regions; exit check against deferred effects; state typing; checking runs
`Step.Observation*`, `Step.Observer*`, `Step.Escape*`	escape and private equivalence; rows modulo the knowledge, full abstraction and congruence; records, escape-relative transport and internal observers; supported probes; local and global matchings; no equivariant selection
`Step.Race*`	the race pair; determinacy; step and history-preserving bisimulation; experimenters, copying and records; resolution of races with records; probabilistic bisimulation
`Step.Model*`	first-order and contextual set models; the proof logic for first-order guards and predicate queries; consistency; guard formulas; cosieves, interiors and forcing; Henkin and free models; controls
`Step.Resolution*`	resolution as a firing; atomicity; soundness and completeness for every selection rule; independence and switching; variant and modulo tabling; the read form
`Step.Superposition*` (ground, firing, guarded, saturation, derivation, generator, higher order, controls)	ground inferences as firings; redundancy as takes; ground completeness; the complete given-clause loop; non-ground firings on the shared store; lifting and non-ground refutational completeness; the Knuth–Bendix ordering; λ-superposition readings

Modules	What they prove
`Step.Search*`	the outcome algebra; budgets and policies; the search combinators; runs as a search space; representatives; the independence check from search; deferred effects; tabling
`Step.Rational*`	rational-tree frames; unitary unification and its computation; the finite fibre; tree model and calculus; the rational step; types keep the occurs check
`Step.Charge*`, `Step.Canonical*`, `Step.Fold*`	conserved charges; canonical interleavings; folding by record automorphisms; the affine pump; the Noether correspondence
`Step.TransfiniteLimits`, `Step.TransfiniteUltrapower`	ω-limits of productive transforms; hyperfinite runs

The cited modules contain no `sorry` and no axiom declaration. The empty-type control at the universe tower assumes cofinally many inaccessible cardinals, as stated there.

19.2 Corrections to earlier statements

- Comparing syntactic label bags is not fully abstract; the fully abstract notion compares rows modulo the knowledge, pointwise in general and uniformly under a principal answer.
- The trace category of the base transport identifies runs that a public frame name separates (`EscapeControls.base_identifies` in `Step.EscapeHelper`); the escape-relative transport keeps them apart.
- Residuals, settlement and the reading of guards were unconstrained by the interface; the laws of lawful transport constrain them, and without them independence depends on the chosen residual.
- Pairwise independence does not give joint firing; cells record it.
- Symbolic solvability accepts holes of empty types; closed asks separate inhabitation from consistency.
- Plain set models are not sound for asks; the free models are the sound and complete class.
- The failure of local matchings to glue without a principal answer is a gap in the order of quantifiers, not contextuality.
- Isomorphisms of states carry a renaming of occurrence identities, and transport holds up to alikeness; equal present sets and equal moved firings are the strict special case.

19.3 Open

Item	What remains
Native transport	isomorphisms as arity-preserving bijections of private declaration names that fix captured and escaped names, acting on rules, outputs, queries, conditional rows and recipes; lawful, natural and escape structure; a diamond of two actual native firings whose clients observe equivalent results
Mixed native templates	publication through producer recipes as recipe instances of the step
Frame change meets conflict	no proved instance yet combines growing frames with takes, reads and predicate queries: the helper instances have none, and the one-frame instance does not grow frames; a dependent instance with them, with state typing
Dependent solver	principal answers or refutation on the dependent pattern fragment; η-long reflection; postponement with twins; the principal pattern answer over the canonical η instance; adequacy of the translation for arbitrary stores

Item	What remains
Admitted object theories	normalization by object-level hereditary substitution; an admitted pattern fibre; reduction of admitted models to canonical ones; an interface obligation that readouts and guards respect the equivalence
Higher cells	lower faces as cells; a precubical set with face maps; coherence of closing isomorphisms for native solver returns
Observation	frame-level full abstraction; interactive congruence relative to records; full abstraction for relational records; the converse of trace refinement of observation; transport indexed by covers of the solutions where no principal answer exists; one structure for escape and row isomorphisms; genuine contextuality with several observers of overlapping parts
Races: readouts	answer bags and the search combinators read only traces: collecting counts runs, is not invariant under renaming and never sees independence
Races: congruence	open bisimilarity depends on the ports of the open system, and closed bisimilarity is not a congruence; a context-labelled step for open bisimilarity
Races: causality	reads, queries and guards: read-then-take is an asymmetric conflict, and guards make history-preserving bisimulation collapse to total order
Races: scope	image-finiteness for writing programs and the Hennessy–Milner characterization of step bisimilarity; weak bisimilarity; probability beyond concrete pairs; the races layer covers only the first-order step
Runtime faithfulness	a runtime must fire exactly the admitted firings, take atomically, run concurrently only what is independent, never reuse identities, and emit what each firing took and made; step bisimilarity of an occurrence-level protocol machine with the step on the independent fragment
Models	a set model of the typed η frames; a proof logic beyond first-order guards and predicate queries; completeness for an axiomatized class (Clark's equality theory [14], Maher's for rational trees); higher-order logic encoded in the step, with soundness, completeness and provenance
Typing	checking with metavariables and postponement; all kernel rules and completeness for fair runs; agreement of runtime acceptance with admission where pending types are inhabited; dependent signatures
Resolution	run-level soundness, completeness, independence and tabling for the read form over growing frames; a metarule match that stays in the pattern fragment
Superposition	the non-ground given-clause loop on the shared step; a premise-copy check in the firing; the full multiset extension of the ordering to literals; guard readings across frames
Search	a finite-classes instance of a concrete step; completion of tables in progress; counting runs per class by groupoid cardinality
Rational trees	substitution on graphs and canonical minimized presentations; soundness of the visited-pair unifier; rational frames with binders
Charges and folding	a computable canonical labelling; uniqueness per trace class; productive folding; the pump for the contextual instance
Transfinite	admission in an ultrapower step as admission at almost every coordinate
Fresh identities	a supply of fresh identities per label as resource data, which would make the bisimulation and the frame rule unconditional; the frame rule lifted to the higher-order step with its side conditions

Vocabulary

frame a declaration family, a formed telescope, or the one object of a substitution action;
solution a morphism out of a frame into a later frame; **closed solution**: one into the closed frame;
space the present occurrences of a state;
information the equation store of a firing at its target frame;
residual a firing read at another firing's result, along that firing's morphism of frames;
alike firings at one state whose data agree up to the frame equivalence;
state class a state up to isomorphism of states;
diamond two firings, each still admitted after the other, with results in one class;
cell a round of firings each admitted, after which the residuals of the others form a cell;
trace a run up to similarity and swaps of independent firings;
record what an observer holds: references, sighted sets, tuples and recorded choices;
recipe publication data fixed before execution: name roles and local conditions of a produced output;
fibre a restriction of the problems (one frame; first order; patterns; simple telescopes) with stronger guarantees;
outcome a multiset of answers with a status, finished or cut off.

References

[1] A. Abel and B. Pientka. Higher-order dynamic pattern unification for dependent types and records. In *TLCA 2011*, LNCS 6690, 2011.

[2] S. Abramsky. Observation equivalence as a testing equivalence. *Theoretical Computer Science* 53(2–3), 1987.

[3] S. Abramsky and A. Brandenburger. The sheaf-theoretic structure of non-locality and contextuality. *New Journal of Physics* 13, 113036, 2011.

[4] P. Aczel. *Non-Well-Founded Sets*. CSLI Lecture Notes 14, 1988.

[5] J. C. Baez and B. Fong. A Noether theorem for Markov processes. *Journal of Mathematical Physics* 54, 013301, 2013.

[6] L. Bachmair and H. Ganzinger. Rewrite-based equational theorem proving with selection and simplification. *Journal of Logic and Computation* 4(3), 1994.

[7] P. Baldan, A. Corradini and U. Montanari. Contextual Petri nets, asymmetric event structures, and processes. *Information and Computation* 171(1), 2001.

[8] J. Barwise and L. Moss. *Vicious Circles*. CSLI Publications, 1996.

[9] A. Bentkamp, J. Blanchette, S. Tourret, P. Vukmirović and U. Waldmann. Superposition with lambdas. *Logical Methods in Computer Science* 17(2), 2021.

[10] H. Betz and T. Frühwirth. A linear-logic semantics for constraint handling rules. In *Principles and Practice of Constraint Programming (CP 2005)*, LNCS 3709, 2005.

[11] R. Bruni and J. Meseguer. Semantic foundations for generalized rewrite theories. *Theoretical Computer Science* 360(1–3), 2006.

[12] A. Schack-Nielsen and C. Schürmann. Celf – a logical framework for deductive and concurrent systems. In *IJCAR 2008*, LNCS 5195, 2008.

[13] W. Chen and D. S. Warren. Tabled evaluation with delays for general logic programs. *Journal of the ACM* 43(1), 1996.

[14] K. L. Clark. Negation as failure. In *Logic and Data Bases*, H. Gallaire and J. Minker (eds.), Plenum, 1978.

[15] K. Watkins, I. Cervesato, F. Pfenning and D. Walker. A concurrent logical framework I: judgments and properties. Technical report CMU-CS-02-101, Carnegie Mellon University, 2002.

[16] A. Nanevski, F. Pfenning and B. Pientka. Contextual modal type theory. *ACM Transactions on Computational Logic* 9(3), 2008.

[17] A. Colmerauer. Prolog and infinite trees. In *Logic Programming*, K. L. Clark and S.-Å. Tärnlund (eds.), Academic Press, 1982.

[18] B. Courcelle. Fundamental properties of infinite trees. *Theoretical Computer Science* 25(2), 1983.

[19] N. Dershowitz and Z. Manna. Proving termination with multiset orderings. *Communications of the ACM* 22(8), 1979.

[20] P. Dybjer. Internal type theory. In *Types for Proofs and Programs (TYPES '95)*, LNCS 1158, 1996.

[21] F. Fages, P. Ruet and S. Soliman. Linear concurrent constraint programming: operational and phase semantics. *Information and Computation* 165(1), 2001.

[22] M. Fiore. Second-order and dependently-sorted abstract syntax. In *LICS 2008*, 2008.

[23] M. Fiore and C.-K. Hur. Second-order equational logic. In *CSL 2010*, LNCS 6247, 2010.

[24] M. Fiore, G. Plotkin and D. Turi. Abstract syntax and variable binding. In *LICS 1999*, 1999.

[25] T. Frühwirth. *Constraint Handling Rules*. Cambridge University Press, 2009.

[26] P. Godefroid. *Partial-Order Methods for the Verification of Concurrent Systems*. LNCS 1032, Springer, 1996.

[27] W. D. Goldfarb. The undecidability of the second-order unification problem. *Theoretical Computer Science* 13(2), 1981.

[28] A. Habel, R. Heckel and G. Taentzer. Graph grammars with negative application conditions. *Fundamenta Informaticae* 26(3–4), 1996.

[29] L. Henkin. Completeness in the theory of types. *Journal of Symbolic Logic* 15(2), 1950.

[30] M. Hennessy and R. Milner. Algebraic laws for nondeterminism and concurrency. *Journal of the ACM* 32(1), 1985.

[31] G. Huet. A unification algorithm for typed λ-calculus. *Theoretical Computer Science* 1(1), 1975.

[32] K. G. Larsen and A. Skou. Bisimulation through probabilistic testing. *Information and Computation* 94(1), 1991.

[33] J.-J. Lévy. *Réductions correctes et optimales dans le λ-calcul*. Thèse d'État, Université Paris 7, 1978.

[34] J. W. Lloyd. *Foundations of Logic Programming*. Second edition, Springer, 1987.

[35] M. J. Maher. Complete axiomatizations of the algebras of finite, rational and infinite trees. In *LICS 1988*, 1988.

[36] A. Mazurkiewicz. Trace theory. In *Advances in Petri Nets 1986*, LNCS 255, 1987.

[37] P.-A. Melliès. Axiomatic rewriting theory I: a diagrammatic standardization theorem. In *Processes, Terms and Cycles*, LNCS 3838, 2005.

[38] J. Meseguer. Conditional rewriting logic as a unified model of concurrency. *Theoretical Computer Science* 96(1), 1992.

[39] J. Meseguer and P. Thati. Symbolic reachability analysis using narrowing and its application to verification of cryptographic protocols. *Higher-Order and Symbolic Computation* 20(1–2), 2007.

[40] D. Miller. A logic programming language with lambda-abstraction, function variables, and simple unification. *Journal of Logic and Computation* 1(4), 1991.

[41] D. Miller. Unification under a mixed prefix. *Journal of Symbolic Computation* 14(4), 1992.

[42] R. Milner. *Communication and Concurrency*. Prentice Hall, 1989.

[43] U. Montanari and F. Rossi. Contextual nets. *Acta Informatica* 32(6), 1995.

[44] T. Murata. Petri nets: properties, analysis and applications. *Proceedings of the IEEE* 77(4), 1989.

[45] F. Pfenning. Unification and anti-unification in the calculus of constructions. In *LICS 1991*, 1991.

[46] A. M. Pitts. *Nominal Sets: Names and Symmetry in Computer Science*. Cambridge University Press, 2013.

[47] A. M. Pitts and I. D. B. Stark. Observable properties of higher order functions that dynamically create local names, or: What's new? In *MFCS 1993*, LNCS 711, 1993.

[48] V. Pratt. Modeling concurrency with geometry. In *POPL 1991*, 1991.

[49] F. Raiser, H. Betz and T. Frühwirth. Equivalence of CHR states revisited. In *CHR 2009*, 2009.

[50] J. A. Robinson. A machine-oriented logic based on the resolution principle. *Journal of the ACM* 12(1), 1965.

[51] C. Rocha, J. Meseguer and C. Muñoz. Rewriting modulo SMT and open system analysis. *Journal of Logical and Algebraic Methods in Programming* 86(1), 2017.

[52] V. A. Saraswat. *Concurrent Constraint Programming*. MIT Press, 1993.

[53] T. Schrijvers, G. Tack, P. Wuille, H. Samulowitz and P. J. Stuckey. Search combinators. *Constraints* 18(2), 2013.

[54] R. J. Simmons. *Substructural Logical Specifications*. PhD thesis, Carnegie Mellon University, 2012.

[55] E. W. Stark. Concurrent transition systems. *Theoretical Computer Science* 64(3), 1989.

[56] H. Tamaki and T. Sato. OLD resolution with tabulation. In *Third International Conference on Logic Programming (ICLP 1986)*, LNCS 225, 1986.

[57] Ultrainfinitism II: infinite wholes and the boundary of effing. Mettapedia Working Notes, 2026.

[58] R. J. van Glabbeek. The linear time – branching time spectrum I: the semantics of concrete, sequential processes. In *Handbook of Process Algebra*, J. A. Bergstra, A. Ponse and S. A. Smolka (eds.), Elsevier, 2001.

[59] R. J. van Glabbeek. On the expressiveness of higher dimensional automata. *Theoretical Computer Science* 356(3), 2006.

[60] R. J. van Glabbeek and U. Goltz. Refinement of actions and equivalence notions for concurrent systems. *Acta Informatica* 37(4–5), 2001.

[61] C. Williams and M. Stay. Native type theory. In *Applied Category Theory 2021*, EPTCS 372, 2022.

[62] G. Winskel. Event structures. In *Advances in Petri Nets 1986*, LNCS 255, 1987.